

ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ РЕСПУБЛИКИ МОЛДОВА

ФАКУЛЬТЕТ МАТЕМАТИКИ И ИНФОРМАТИКИ

ДЕПАРТАМЕНТ ИНФОРМАТИКИ

Цуркану Элина

Разработка образовательного приложения с использованием React, Hibernate, Spring

Прикладная информатика

Дипломная работа

Директор Департамента: _____ Капчеля Титу, доцент, доктор наук

Научный руководитель: _____ Епифанова Ирина, магистр информатики

Автор: _____ Цуркану Элина, IA1902

Кишинев, 2022

Содержание

СПИСОК ПРИНЯТЫХ СОКРАЩЕНИЙ	3
АННОТАЦИЯ	5
ВВЕДЕНИЕ	8
I. ОПИСАНИЕ ТЕОРЕТИЧЕСКОЙ ЧАСТИ.....	11
1.1 Описание ORM технологий.....	11
1.2 Сравнительный анализ JS фреймворков	13
1.3. Детальное описание React JS	15
1.4 Изучение фреймворка Hibernate	16
1.5 Исследование фреймворка Spring.....	21
II. ДОПОЛНИТЕЛЬНЫЕ ПРИЛОЖЕНИЯ, ИСПОЛЬЗОВАННЫЕ В РАБОТЕ	23
1.1 Описание СУБД MySQL	23
1.2 Обзор среды разработки IntelliJ IDEA.....	23
1.3 Описание графического редактора Adobe Photoshop	24
1.4 Описание видеоредактора Movavi Video Editor	25
III. ОПИСАНИЕ ПРАКТИЧЕСКОЙ ЧАСТИ.....	26
3.1 Анализ существующих обучающих игр.....	26
3.2 Диаграмма компонентов	30
3.3 Диаграмма прецедентов использования.....	31
3.4 ER Диаграмма.....	34
3.5 Диаграммы последовательностей.....	36
3.6 Дерево интерфейсов.....	37
ЗАКЛЮЧЕНИЕ	43
БИБЛИОГРАФИЯ	45
ПРИЛОЖЕНИЕ.....	47

СПИСОК ПРИНЯТЫХ СОКРАЩЕНИЙ

Аббревиатура	Расшифровка аббревиатуры
CSS	Cascading Style Sheets - набор параметров форматирования, который применяется к элементам документа, чтобы изменить их внешний вид.
JS	Javascript - мультипарадигменный язык программирования. Поддерживает объектно-ориентированный, императивный и функциональный стили.
СУБД	Система управления базами данных – это комплекс программно-языковых средств, позволяющих создать базы данных и управлять данными.
PC	Personal Computer - персональный компьютер.
ORM	Object-Relational Mapping - технология программирования, которая связывает базы данных с концепциями объектно-ориентированных языков программирования, создавая «виртуальную объектную базу данных».
ER	Entity-Relationship Diagram - Диаграмма отношений сущностей - это визуальное представление базы данных, которое показывает, как связаны элементы внутри.
СКС	Синдром компьютерного стресса - это реакция организма человека на длительную работу за компьютером.
SQL	Structured Query Language - декларативный язык программирования, применяемый для создания, модификации и управления данными в

	реляционной базе данных, управляемой соответствующей системой управления базами данных.
CoRM	Collection-Relational Mapping - реляционное отображение коллекций.
UI	User Interface - дизайн пользовательского интерфейса.
UML	Unified Modeling Language - язык графического описания для объектного моделирования в области разработки программного обеспечения, для моделирования бизнес-процессов, системного проектирования и отображения организационных структур.
ООП	Объектно-ориентированное программирование - методология программирования, основанная на представлении программы в виде совокупности объектов, каждый из которых является экземпляром определённого класса, а классы образуют иерархию наследования.
XML	eXtensible Markup Language - расширяемый язык разметки.

АННОТАЦИЯ

Тема работы:

“Разработка образовательного приложения с использованием React, Hibernate, Spring.”

Автор работы:

Цуркану Элина, факультет Математики и Информатики, Департамент Информатики, специальность Прикладная Информатика, группа: IA1902.

Руководитель:

Епифанова Ирина, магистр информатики.

Целью работы является создание образовательного приложения с использованием фреймворков React, Hibernate и Spring.

В представленной работе был показан процесс постепенной реализации проекта.

Дипломная работа состоит из введения, 3 глав и заключения. В первой главе описана теоретическая часть. Во второй главе рассмотрены приложения, использованные в работе. Третья глава посвящена описанию практической части. В заключении подведены итоги теоретических исследований и общие выводы по проделанной практической работе, а также описаны планы по дальнейшему развитию проекта.

Дипломная работа состоит из 46 страниц и содержит 19 рисунков, 2 таблицы.

ADNOTARE

Tema de lucru:

„Elaborarea aplicații educaționale utilizând React, Hibernate, Spring.”

Autorul lucrării:

Țurcanu Elina, Facultatea de Matematică și Informatică, Departamentul de Informatică, specialitatea Informatică Aplicată, grup: IA1902.

Conducător științific:

Epifanova Irina, maestru în informatică.

Scopul lucrării este de a crea o aplicație educațională folosind tehnologii React, Hibernate și Spring.

În lucrarea curentă a fost prezentat procesul de implementare treptată a proiectului.

Teza constă din introducere, 3 capitole și concluzii. Primul capitol descrie partea teoretică. Al doilea capitol se discută instrumentele soft existente utilizate în dezvoltarea părții practice. Al treilea capitol este dedicat descrierii părții practice. În concluzii sunt descrise rezultatele cercetărilor teoretice și planurile dezvoltării proiectului în viitor.

Teza este formată din 46 de pagini și conține 19 de desene, 2 tabele.

ANNOTATION

Topic of work:

" Educational application's development based on React, Hibernate, Spring."

The author of the work:

Tsurcanu Elina, Faculty of Mathematics and Informatics, Department of Informatics, specialty Applied Informatics, group: IA1902.

Scientific director:

Epifanova Irina, master of computer science.

The concept of the thesis is to create an educational application using the React, Hibernate and Spring frameworks.

In the presented work, the process of gradual implementation of the project was shown.

The thesis consists of an introduction, 3 chapters and a conclusion. The first chapter describes the theoretical part. The second chapter discusses the applications used in the project. The third chapter is devoted to the description of the practical part. The conclusions describe the results of theoretical research and plans for future project development.

The thesis consists of 46 pages and contains 19 images, 2 tables.

ВВЕДЕНИЕ

В современном мире дети очень много времени уделяют компьютеру, смартфонам, планшетам. Учителя и родители всеми способами пытаются отвлечь детей от вышеприведенных электронных устройств, так как они негативно влияют на здоровье человека.

Среди пользователей РС выявлен новый тип заболевания - синдром компьютерного стресса (СКС), который проявляется головной болью, воспалением слизистой оболочки глаз, повышенной раздражительностью, вялостью и депрессией.

Причинами разнообразных симптомов СКС, по мнению исследователей, являются:

- сверхнапряженная работа глаз и неправильное положение тела;
- ношение несоответствующих очков или контактных линз;
- неправильная организация рабочего места;
- суммирование физических, умственных и визуальных нагрузок;
- низкий уровень визуальной подготовленности для работы с компьютером.

В таблице 1 приведены данные о количестве людей, обратившихся с признаками компьютерной болезни. Эти исследования проведены медиками центральной районной больницы г. Тарко-Сале в России. [1, стр..10-20]

Однако, в связи с недавней глобальной пандемией, мобильные устройства и РС стали неотъемлемой частью образования. Было создано достаточно большое количество образовательных приложений, позволяющих упростить, оптимизировать, и, самое главное, сделать процесс обучения гораздо более увлекательным и интересным.

Образовательное приложение - это часть программного обеспечения, которое облегчает и поддерживает онлайн-обучение, особенно самообучение.

По сути, образовательные приложения предназначены для того, чтобы сделать образование более доступным для всех. Искусственный интеллект, машинное обучение, аналитика данных, расширенная реальность и технологии 5G позволяют многим разработчикам приложений организовывать образовательный процесс в любой части земного шара. Преподаватели могут использовать образовательное приложение, чтобы помочь ученикам и студентам с любым типом дистанционного обучения. В современном мире не только учащиеся, но даже профессионалы пользуются образовательными приложениями.

Таблица 1. Количество людей, обратившихся с признаками компьютерной болезни [1]

Симптомы воздействия компьютера	Процент операторов, сообщивших о симптомах			
	Неполная смена. Работа за дисплеями до 12 месяцев	Полная смена Работа за дисплеями до 12 месяцев	Работа за дисплеями более 12 месяцев	Работа за дисплеями более 2 лет
Головная боль и боль в глазах	8	35	51	76
Утомление и головокружение	5	32	41	69
Нарушение ночного сна	-	8	15	50
Сонливость в течение дня	11	22	48	76
Изменения настроения	8	24	27	50
Повышенная раздражительность	3	11	22	51
Депрессия	3	16	22	50
Снижение интеллектуальных способностей, ухудшение памяти	-	3	12	40
Натяжение кожи лба и головы	3	51	13	19
Выпадение волос	-	-	3	5
Боль в мышцах	11	14	21	32
Боль в области сердца, одышка, учащенное сердцебиение	-	5	7	32

Образовательные приложения затрагивают множество областей: математика, физика, история, литература и т.д. Но, так как профиль данной работы связан с IT сферой, здесь будут рассматриваться только приложения, обучающие детей школьного возраста программированию в игровой форме, так как многие дети начинают интересоваться современными технологиями и программированием еще с раннего возраста.

Автор данной работы проанализировал и протестировал существующие приложения, обучающие программированию, вместе с учениками академии “IT Step”, где автор работает преподавателем информатики на протяжении 3 лет.

В результате проведенных исследований были выявлены некоторые недочеты существующих приложений, которые усложняют процесс обучения.

В связи с этим, было принято решение создать собственное приложение, обучающее детей программированию в легкой интуитивной игровой форме.

Теоретические цели работы:

- Исследование существующих ORM технологий и их сравнительный анализ.
- Исследование существующих фреймворков для языка программирования JS
- Детальное изучение фреймворков React, Hibernate и Spring и интеграция их в проект
- Изучение структуры образовательных приложений
- Сравнительный анализ существующих образовательных приложений

Практические задачи работы:

- Создание проекта собственного образовательного приложения, позволяющего включить все изученные технологии.
- Реализация на Java и JS спроектированного приложения
- Тестирование приложения вместе с учениками академии “IT Step”

I. ОПИСАНИЕ ТЕОРЕТИЧЕСКОЙ ЧАСТИ

1.1 Описание ORM технологий

ORM (англ. object-relational mapping, рус. объектно-реляционное отображение) - технология программирования, которая связывает базы данных с концепциями объектно-ориентированных языков программирования, создавая «виртуальную объектную базу данных». Существуют как проприетарные, так и свободные реализации этой технологии.[5]

Принцип работы ORM

Ключевой особенностью ORM является отображение, которое используется для привязки объекта к его данным в БД. ORM создает «виртуальную» схему базы данных в памяти и позволяет манипулировать данными уже на уровне объектов. Отображение показывает, как объект и его свойства связаны с одной или несколькими таблицами и их полями в базе данных. ORM использует информацию этого отображения для управления процессом преобразования данных между базой и формами объектов, а также для создания SQL-запросов для вставки, обновления и удаления данных в ответ на изменения, которые приложение вносит в эти объекты.

Преимущества и недостатки использования ORM

Использование ORM в проекте избавляет разработчика от необходимости работы с SQL и написания большого количества кода, часто однообразного и подверженного ошибкам. Весь генерируемый ORM код проверен и оптимизирован, поэтому не нужно задумываться о его тестировании. В то же время при использовании этой технологии происходит потеря производительности. Это происходит потому, что большинство ORM предназначены для обработки широкого спектра сценариев использования данных, гораздо большего, чем любое отдельное приложение когда-либо сможет использовать. Поэтому приходится выбирать, что более приоритетно - удобство или производительность. Конечно, работа с БД посредством грамотно написанного SQL-кода будет намного эффективнее, но не стоит забывать и о таком параметре, как время - то, что с легкостью пишется с использованием ORM за неделю, можно реализовывать ни один месяц собственными усилиями. Кроме того, большинство современных ORM позволяют программисту при необходимости самому задавать код SQL-запросов. Без сомнений, для небольших проектов использование ORM будет куда более оправдано, чем разработка собственных библиотек для работы с Базой данных.

Альтернативы использования ORM

Среди аналогов технологии ORM можно выделить следующие:

- денормализация - сознательное нарушение нормализации таблиц в реляционной модели, которое, хотя и приводит к избыточности данных наряду с

появлением необходимости их синхронизации, обеспечивает преимущество ускорения доступа к данным;

- подход CoRM (Collection-Relational Mapping - реляционное отображение коллекций), при котором осуществляется объединение разрозненных коллекций объектов с помощью хорошо определенных реляционных взаимоотношений между коллекциями и прототипом которого могут служить документ-ориентированные СУБД (например, MongoDB). В случае CoRM отсутствует ограничение на возможность хранить состояние разных объектов одного класса в разных коллекциях и ограничение на одновременное хранение в коллекции объектов, которые принадлежат разным классам.

От обычного ORM-подхода реляционное отображение коллекций отличается тем, что коллекция напрямую не привязана к классу и, в теории, может включать в себя любой объект, в случае соблюдения некоторых минимальных требований к данному объекту, например, требование наличия способности к сериализации). Однако к этим требованиям не относятся ограничения на структуру объекта либо использование особых типов данных.

Реализации ORM

Hibernate - фреймворк для языка программирования Java, предназначенная для решения задач объектно-реляционного отображения (ORM), самая популярная реализация спецификации JPA.

EclipseLink - это свободный фреймворк для языка программирования Java, предназначенный для решения задач объектно-реляционного отображения ORM. Разрабатывается Фондом Eclipse (Eclipse Foundation). Позволяет работать с разными сервисами данных, включая базы данных, веб-сервисы, Object XML mapping (OXM), и корпоративные информационные службы.

Doctrine - объектно-реляционный проектор (ORM) для PHP 7.1+, который базируется на слое абстракции доступа к БД (DBAL). Одной из ключевых возможностей Doctrine является запись запросов к БД на собственном объектно-ориентированном диалекте SQL, называемом DQL (Doctrine Query Language) и базирующемся на идеях HQL (Hibernate Query Language).

Tryton - высокоуровневая платформа для разработки приложений, использующая трехуровневую архитектуру, на основании которой создано бизнес-решение (или ERP), представленное с помощью так называемых модулей Tryton. [5]

1.2 Сравнительный анализ JS фреймворков

Существует множество Javascript фреймворков, выпускаемых и обновляемых ежедневно, и иногда бывает сложно идти в ногу со временем и понимать, какой из фреймворков подходит для проекта лучше всего.

Прежде чем выбрать какой-либо из них для веб-приложения, необходимо учитывать множество факторов, например, конкретные требования проекта или сильные и слабые стороны фреймворка.

Перед началом реализации проекта необходимо предварительно провести анализ, чтобы выбрать именно ту технологию, которая лучше подойдет.

Преимущества фреймворков

Javascript фреймворк - это набор методов и функций, которые написаны на языке Javascript и готовы к использованию разработчиками. Эти функции упрощают взаимодействие веб-приложения с сервером и ускоряют манипулирование элементами на веб-странице или в приложении.

Основным преимуществом использования фреймворков является тот факт, что они обеспечивают мгновенную обратную связь с теми, кто в настоящее время взаимодействует с сайтом. На традиционных веб-сайтах без фреймворков первоначальный контент хранится на сервере, и любой новый контент, который необходимо загрузить, требует перезагрузки страницы. А при использовании фреймворком перезагружаются только необходимые блоки веб-сайта.

Такой подход сейчас используют социальные сети, например, Facebook. Если вы находитесь на странице «Новости» и переходите на страницу «Сообщения», перезагрузки страницы не происходит, вместо этого меняется несколько блоков на сайте.

Ниже упомянуты некоторые из наиболее примечательных преимуществ использования фреймворков:

- **Стоимость.** Стоимость разработки приложений для веб-сайтов снижается благодаря Javascript фреймворкам, поскольку они бесплатны и имеют открытый исходный код.
- **Скорость разработки.** У Javascript фреймворков хорошая документация (описание работы функций со своеобразной «книгой рецептов»), множество форумов и групп поддержки. Некоторые из Javascript-фреймворков поддерживаются такими известными компаниями, как Google или Facebook. Благодаря такому количеству информации увеличивается скорость разработки.
- **Эффективность.** Использование предварительно созданных функций и шаблонов позволяет реализовывать проекты более качественно. Разработчики

пишут меньше кода, что приводит к более быстрому и эффективному выполнению проектов.

Популярные фреймворки

Javascript фреймворки являются одной из наиболее предпочтительных платформ для создания современных одностраничных веб-приложений, социальных сетей, eCommerce продуктов, SaaS платформ и многого другого.

В итоге, разработка современного веба стала практически невозможна без использования фреймворков. Конечно, еще остались компании, которые используют классический Javascript, но в таких случаях скорость и стоимость разработки увеличивается в разы.

Как следствие сложности классического JS, на рынке появились сотни фреймворков. Но уверенное лидерство на рынке разработки заняла тройка: Angular, React и Vue.

Angular

Angular - это кроссплатформенный фреймворк, который придерживается MVC-шаблона проектирования и поощряет слабую связь между представлением, данными и логикой компонентов (составных частей приложения).

Angular переносит на клиентскую сторону часть серверных служб. Следовательно, уменьшается нагрузка на сервер и веб-приложение становится легче.

Благодаря TypeScript, код проектов становится чистый, удобный для понимания разработчиков и содержит меньше ошибок

Сильные стороны Angular: отличная документация, поддержка компанией Google, огромный набор инструментов для разработки (Material UI, CLI и т.д.)

Одной из слабых сторон является высокий порог вхождения в проекты для разработчиков, потому что, например, нужно знать TypeScript. Следовательно, это усложняет разработку проектов, особенно если проект передается от одной команды к другой.

Вторая проблема Angular - очень частый релиз новых версий. В июне 2019 года вышла уже 8-я версия фреймворка. Этот факт также говорит о том, что проекты на Angular сложнее поддерживать.

React.js

React - это JavaScript библиотека с открытым исходным кодом для разработки пользовательских интерфейсов. Впервые React был использован в 2011 году в ленте Facebook, затем в 2012 в ленте Instagram.

React до сих пор разрабатывается и поддерживается Facebook и Instagram. React может использоваться не только для браузерных веб-приложений, но также для мобильных

приложений. Цель React - предоставить высокую скорость, простоту и работоспособность приложений на разных платформах.

Но React - это не полноценный фреймворк, а библиотека функций. Чтобы использовать его как фреймворк, нужно дополнительно подключать сторонние библиотеки.

Сильные стороны React: скорость работы, легковесность, кроссплатформенность и большое сообщество.

Слабой стороной является тот факт, что для полноценной работы нужны сторонние Javascript-библиотеки, что усложняет процесс разработки. Вторым минусом библиотеки - это отсутствие следования стандартам в написании кода на HTML и CSS, какое есть, например, у Angular и Vue.js.

Vue.js

Vue - это прогрессивный фреймворк для создания пользовательских интерфейсов. Vue создан пригодным для постепенного внедрения, в отличие от Angular или React. Это значит, что внедрять этот фреймворк можно поэтапно начиная с определенных страниц, что значительно упрощает разработку.

Создателем Vue.js является Иван Йу. Vue широко используется среди китайских компаний, например, Alibaba, Baidu, Xiaomi и др. Недавно система управления репозиториями GitLab тоже перешла на Vue.js.

Vue в первую очередь решает задачи уровня представления (view), что упрощает интеграцию с другими библиотеками и существующими проектами.

Vue вобрал в себя лучшие стороны Angular и React: скорость, легковесность, возможность поддержки таких технологий, как TypeScript и JSX. Но, при этом, Vue остался верен стандартам написания кода на HTML и CSS, что облегчает процесс разработки и поддержки проекта.

Таким образом, для любых разработчиков, которые знают основы frontend-технологий не будет проблемой разработать или взять на поддержку проекты на Vue. [6]

1.3. Детальное описание React JS

React JS – это фреймворк, который используется для создания динамических веб и мобильных приложений. React - это инструмент для создания пользовательских интерфейсов. Его главная задача - обеспечение вывода на экран того, что можно видеть на веб-страницах. React значительно облегчает создание интерфейсов благодаря разбиению каждой страницы на небольшие фрагменты. [7]

Помимо универсальности React JS обладает рядом преимуществ перед другими системами. В частности следует отметить такие качества как:

1. *Виртуальная объектная модель документа.*

Вместо медленных и неудобных взаимодействий непосредственно с реальной объектной моделью документа, React взаимодействует с ее облегченной копией - виртуальной DOM, поэтому реальная DOM обновляется только после взаимодействия с виртуальной DOM.

2. *Повторное применение компонентов.*

При работе с ReactJS создаются многоразовые компоненты: чаще всего, компонент пользовательского интерфейса можно использовать в других частях кода или даже в разных проектах практически без изменений.

Более того, разработчикам React-приложений доступны библиотеки готовых компонентов с открытым исходным кодом. Благодаря им время разработки сокращается, что очень важно для стартапов, которым необходимо экономить не только деньги, но и время.

3. *Нисходящий поток данных.*

В React-приложении данные между элементами передаются только одним способом. Нисходящий поток данных предотвращает ошибки, облегчает отладку.

4. *Огромное сообщество.*

Сообщество React предоставляет тонны ценных библиотек, облегчающих процесс разработки, открыты новые способы и возможности улучшить качество кода приложений.[8]

1.4 Изучение фреймворка Hibernate

Hibernate – это ORM фреймворк для Java с открытым исходным кодом. Эта технология является крайне мощной и имеет высокие показатели производительности.

Hibernate создаёт связь между таблицами в базе данных (далее – БД) и Java-классами и наоборот. Это избавляет разработчиков от огромного количества лишней, рутинной работы, в которой крайне легко допустить ошибку и крайне трудно потом её найти. [9]

Преимущества:

- Устраняет множество повторяющегося кода, который постоянно преследует разработчика при работе с JDBC. Скрывает от разработчика множество кода, необходимого для управления ресурсами и позволяет сосредоточиться на бизнес логике.

- Поддерживает XML так же, как и JPA аннотации, что позволяет сделать реализацию кода независимой.
- Предоставляет собственный мощный язык запросов (HQL), который похож на SQL. Стоит отметить, что HQL полностью объектно-ориентирован и понимает такие принципы, как наследование, полиморфизм и ассоциации (связи).
- Hibernate легко интегрируется с другими Java EE фреймворками, например, Spring Framework поддерживает встроенную интеграцию с Hibernate.
- Поддерживает ленивую инициализацию используя прокси объекты и выполняет запросы к базе данных только по необходимости.
- Поддерживает разные уровни cache, а, следовательно, может повысить производительность.
- Важно, что Hibernate может использовать чистый SQL, а значит поддерживает возможность оптимизации запросов и работы с любым сторонним вендором БД.
- Hibernate - open source проект. Благодаря этому доступны тысячи открытых статей, примеров, а также документации по использованию фреймворка. [10]

Проект Hibernate имеет следующую структуру (Рис.1.4.1):

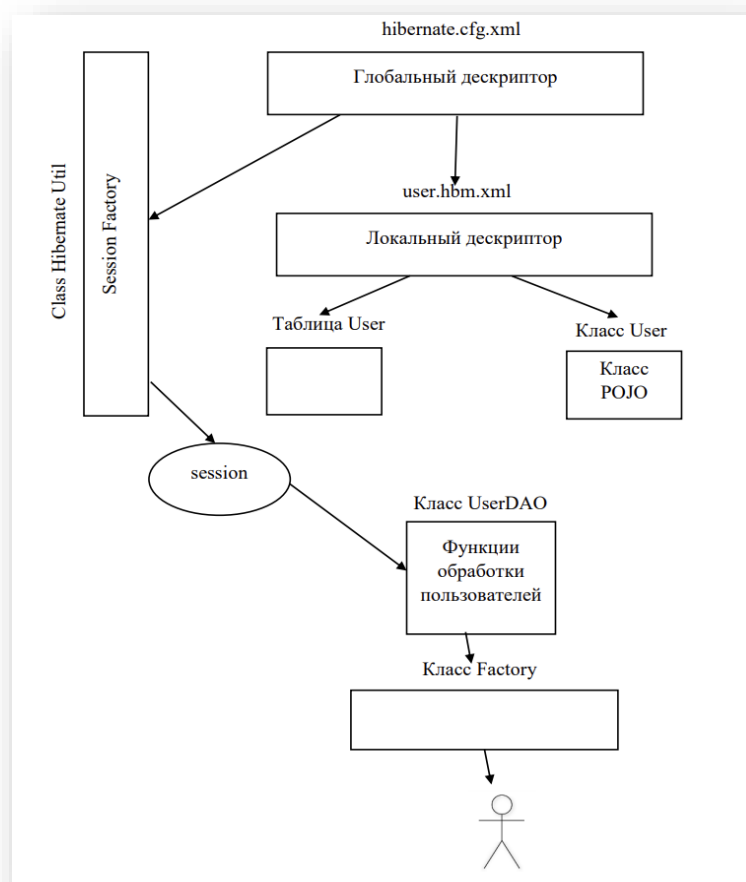


Рис. 1.4.1. Структура проекта Hibernate

Во-первых, необходимо описать классы сущностей. Для примера был взят класс User, где хранится вся информация о пользователе. Все классы сущностей должны соответствовать Java naming conventions, т.е. у них должны быть обязательно функции get и set, и конструктор по умолчанию.

```
public class User {  
    ...  
    //параметры  
  
    public User() {}  
    ...  
    //функции get и set для всех параметров  
}
```

Далее для классов необходимо описать маппинг в виде xml-файлов. Эти файлы будут отвечать за взаимодействие объектов с Hibernate и с базой данных.

```
User.hbm.xml  
<hibernate-mapping>  
    <class name=«logic.User» table=«user»>  
        <id column=«user_id» name=«id» type=«java.lang.Long»>  
            <generator class=«increment»/>  
        </id>  
        <property column=«number» name=«number» type=«java.lang.String»/>  
  
        <set name=«parent_child» table=«parent_child» lazy=«false»>  
            <key column=«user_id»/>  
            <many-to-many column=«connection_id» class=«logic.parent_child»/>  
        </set>  
  
    </class>  
</hibernate-mapping>
```

Тег class имеет два параметра: параметр name - Имя класса (необходимо указывать полный путь с учетом структуры пакетов) и параметр table - имя таблицы в базе данных, на которую будет маппиться наш класс.

Тег id описывает идентификатор. Параметр column указывает на какую колонку в таблице будет ссылаться поле id объекта, так же нужно указать класс и указать generator, который отвечает за генерацию id.

Тег `property` описывает простое поле объекта, в качестве параметров указываем имя поля, его класс и имя колонки в таблице.

Тег `set` описывает поле в котором содержится некий набор(коллекция) объектов. Тег содержит параметр `name` - имя поля объекта, параметр `table` — имя таблицы связи(в случае отношения многие ко многим) и параметр `lazy`. Когда в параметре `lazy` указывается значение `false`, то при получении объекта `parent_child` из базы вместе с объектом достается и коллекция объектов `User`, так как `user` это поле объекта `parent_child`. А если в качестве параметра указывается значение `true`, то коллекция объектов `User` не вытаскивается, для ее получения надо явно вызывать необходимый метод. Тег `key` имеет параметр `column`, который говорит, на какую колонку в таблице связи будет ссылаться поле нашего объекта.

Тег `many-to-many` описывает связь типа многие ко многим, в качестве параметров тег использует `column` - имя колонки второй колонки в таблице связи и параметр `class`, указывающий какого класса будут объекты на той стороне.

Далее необходимо создать главный конфигурационный файл `hibernate.cfg.xml` - файл, откуда будет собираться вся необходимая информация.

```
<hibernate-configuration>

<session-factory>

  <property name=«connection.url»>jdbc:mysql://localhost/symonscat</property>
  <property name=«connection.driver_class»>com.mysql.jdbc.Driver</property>
  <property name=«connection.username»>root</property>
  <property name=«connection.password»/>
  <property name=«connection.pool_size»>1</property>
  <property name=«current_session_context_class»>thread</property>
  <property name=«show_sql»>true</property>
  <property name=«dialect»>org.hibernate.dialect.MySQL5Dialect</property>
  <mapping resource=«logic/User.hbm.xml»/>
</session-factory>

</hibernate-configuration>
```

Далее нужно создать класс, который будет использовать конфиг-файл и возвращать объект типа `SessionFactory`, который отвечает за создание `hibernate`-сессии.

```

public class HibernateUtil {
    private static final SessionFactory sessionFactory;
    static {
        try {
            sessionFactory = new Configuration().configure().buildSessionFactory();
        } catch (Throwable ex) {
            System.err.println(«Initial SessionFactory creation failed.» + ex);
            throw new ExceptionInInitializerError(ex);
        }
    }
    public static SessionFactory getSessionFactory() {
        return sessionFactory;
    }
}

```

Далее нужно установить взаимодействие приложения с базой данных. Для этого для каждого класса-сущности, нужно определить интерфейс, содержащий набор необходимых методов.

```
package DAO;
```

```
import logic.User;
```

```
...
```

```

public interface UserDAO {
    public void addUser(User user) throws SQLException;
    public void updateUser(Long user_id, User user) throws SQLException;
    public User getUserById(Long user_id) throws SQLException;
    public Collection getAllUsers() throws SQLException;
    public void deleteUser(User user) throws SQLException;
}

```

Также необходимо определить реализацию вышеописанного интерфейса в классе UserDAOImpl

Далее нужно создать класс Factory, к которой есть возможность обращаться за реализациями DAO, от которых и могут быть вызваны необходимые методы. [21]

```

public class Factory {

    private static UserDao userDao = null;
    private static Factory instance = null;

    public static synchronized Factory getInstance(){
        if (instance == null){
            instance = new Factory();
        }
        return instance;
    }

    public UserDao getUserDao(){
        if (userDao == null){
            userDao = new UserDaoImpl();
        }
        return userDao;
    }

}

```

1.5 Исследование фреймворка Spring

Spring Framework - универсальный фреймворк с открытым исходным кодом для Java-платформы. По сути Spring Framework представляет собой просто контейнер внедрения зависимостей, с несколькими удобными слоями. Это все позволяет разработчику быстрее и удобнее создавать Java-приложения.

Первая версия была написана Родом Джонсоном, который впервые опубликовал её вместе с изданием своей книги «Expert One-on-One Java EE Design and Development» (Wrox Press, октябрь 2002 года).

Несмотря на то, что Spring не обеспечивал какую-либо конкретную модель программирования, он стал широко распространённым в Java-сообществе главным образом как альтернатива и замена модели Enterprise JavaBeans. Spring предоставляет большую свободу Java-разработчикам в проектировании; кроме того, он предоставляет хорошо документированные и лёгкие в использовании средства решения проблем, возникающих при создании приложений корпоративного масштаба.

Между тем, особенности ядра Spring (Рис. 2.3.1) применимы в любом Java-приложении, и существует множество расширений и усовершенствований для построения веб-приложений на Java Enterprise платформе. По этим причинам Spring приобрёл большую популярность и признаётся разработчиками как стратегически важный фреймворк. [2, стр.31-40]

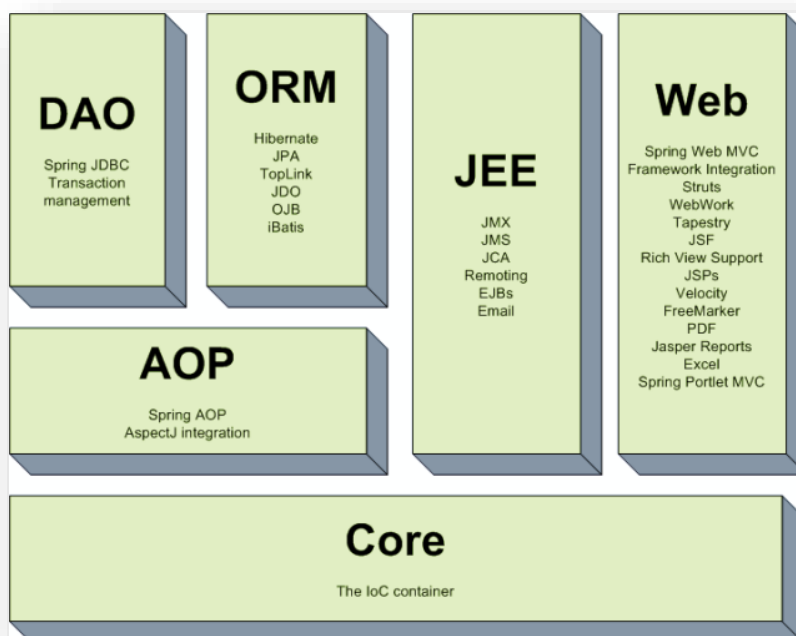


Рис. 1.5.1. Схема ядра Spring

Основное преимущество фреймворка Spring - возможность разработки приложения как набора слабосвязанных (loose-coupled) компонентов. Чем меньше компоненты приложения знают друг о друге, тем проще разрабатывать новый и поддерживать существующий функционал приложения. Классический пример - управление транзакциями. Spring позволяет вам управлять транзакциями совершенно независимо от основной логики взаимодействия с БД. Изменение этой логики не порушит транзакционность, равно как изменение логики управления транзакциями не сломает логику программы. Spring поощряет модульность. Компоненты можно добавлять и удалять (почти) независимо друг от друга. В принципе, приложение можно разработать таким образом, что оно даже не будет знать, что управляется Spring'ом.[11]

II. ДОПОЛНИТЕЛЬНЫЕ ПРИЛОЖЕНИЯ, ИСПОЛЬЗОВАННЫЕ В РАБОТЕ

При создании практической части данной работы были также использованы следующие существующие soft продукты:

- Для реализации и управления базы данных – СУБД MySql
- В качестве среды разработки – IntelliJIdea
- Для создания или редактирования изображений – Adobe Photoshop
- Для редактирования видео – Movavi Video Editor

1.1 Описание СУБД MySQL

Одной из самых популярных СУБД на сегодняшний день является MySQL, распространяемая свободно (с некоторыми ограничениями). Эта серверная система способна эффективно функционировать во взаимодействии с интернет-сайтами и веб-приложениями.

Несмотря на отсутствие некоторого функционала, имеющегося у других СУБД, MySQL обладает достаточно обширным разнообразием доступных инструментов для создания приложений.

Помимо универсальности и распространенности СУБД MySQL обладает целым комплексом важных преимуществ перед другими системами. В частности следует отметить такие качества как:

- Многопоточность, поддержка нескольких одновременных запросов.
- Обширный функционал. Система MySQL обладает практически всем необходимым инструментарием, который может понадобиться в реализации практически любого проекта.
- Безопасность. Система изначально создана таким образом, что множество встроенных функций безопасности в ней работают по умолчанию.
- Масштабируемость. Являясь весьма универсальной СУБД, MySQL в равной степени легко может быть использована для работы и с малыми, и с большими объемами данных.
- Скорость. Высокая производительность системы обеспечивается за счет упрощения некоторых используемых в ней стандартов. [12]

1.2 Обзор среды разработки IntelliJ IDEA

IntelliJ IDEA - интегрированная среда разработки программного обеспечения для многих языков программирования, в частности Java, JavaScript, Python, разработанная компанией JetBrains. IntelliJ IDEA представляет собой высокотехнологичный комплекс

тесно интегрированных инструментов программирования, включающий интеллектуальный редактор исходных текстов с развитыми средствами автоматизации, мощные инструменты рефакторинга кода, встроенную поддержку технологий J2EE, механизмы интеграции со средой тестирования Ant/JUnit и системами управления версиями, уникальный инструмент оптимизации и проверки кода Code Inspection, а также инновационный визуальный конструктор графических интерфейсов. [13]

Уникальные возможности JetBrains IntelliJ IDEA избавляют программиста от груза рутинной работы, помогают своевременно устранить ошибки и повысить качество кода, поднимая продуктивность разработчика на новую высоту.

Преимущества IntelliJ IDEA:

- Все необходимые инструменты уже под рукой, а значит не нужно искать и устанавливать плагины для интеграции с системами контроля версий и поддержки популярных языков и фреймворков.
- Умное автодополнение кода. В то время как базовое автодополнение предлагает имена классов, методов, полей и ключевых слов в области видимости, умное автодополнение предлагает только те элементы кода, которые актуальны в текущем контексте.
- Продуктивная работа. IntelliJ IDEA анализирует однообразные задачи в процессе разработки и автоматизирует их, чтобы сосредоточиться на общей картине. [14]

1.3 Описание графического редактора Adobe Photoshop

Программа Adobe Photoshop CC — это новые возможности работы с цифровыми изображениями и фотографиями, обеспечивающие интеллектуальное ретуширование, реалистическое раскрашивание и выделение изображений с поддержкой 64-разрядных вычислений и широким набором улучшенных рабочих процессов, повышающих продуктивность.

Adobe Photoshop предназначен для обработки и создания точечной (растровой) графики (bitmapped images). Программа используется для работы с фотографиями и коллажами из них, рисованными иллюстрациями, слайдами и мультипликацией, изображениями для веб-страниц, кинокадрами.

Photoshop обладает практически безграничными возможностями. Его с успехом используют фотохудожники для обработки снимков, программа позволяет проводить ретушь, цветовую и тоновую коррекцию, осуществлять размытие и повышение резкости. Возможность выделения и работы с частями изображения незаменима для оформления монтажей.

Обширный набор специальных фильтров (искажения, цветовые сдвиги, другие специальные эффекты) активно используется при создании как коммерческого дизайна, так и художественных произведений.

Мультипликаторы, специалисты по созданию сайтов найдут палитру для удобного и полного впечатляющих возможностей интерфейса программы Photoshop.

Программа предоставляет весь спектр возможностей для допечатного процесса — от сканирования до установки параметров цветоделения и растривания. [4, стр.19]

1.4 Описание видеоредактора Movavi Video Editor

Movavi Video Editor – универсальный инструмент для работы с видео. Этот простой и функциональный видеоредактор позволяет монтировать, улучшать качество видео, применять спецэффекты, добавлять титры и музыку, настраивать переходы между видеофрагментами, создавать слайд-шоу, разрезать и объединять файлы, озвучивать проекты в режиме реального времени, сохранять ролики в популярные форматы, делиться ими онлайн и многое другое. Все инструменты редактирования доступны как для обычного, так и для 3D-видео.

Основные возможности продукта:

- Редактирование видеофайлов в 2D и 3D на монтажном столе. Наложение субтитров и музыки. Спецэффекты и фильтры для улучшения качества изображения.
- Эффект «Хромакей» позволяет заменить однотонный фон ролика любым живописным видео.
- Отдельные дорожки для видео и изображений, звука и титров делают работу с разными типами медиафайлов проще.
- Добавляйте в саундтрек записи с микрофона, электронных музыкальных инструментов или другого входа звуковой карты.
- Работайте с HD-видео быстрее: при загрузке “тяжелого” видео в Movavi Video Editor программа создает специальный видеофайл с меньшим, чем у исходного, разрешением.
- Встроенная программа для разрезания файлов помогает легко и быстро разделить видео на несколько фрагментов.
- Специальный режим Сценария подойдет для создания слайд-шоу.

[20]

III. ОПИСАНИЕ ПРАКТИЧЕСКОЙ ЧАСТИ

3.1 Анализ существующих обучающих игр

На сегодняшний день уже было создано достаточно большое количество игр, которые обучают детей и подростков программировать. В ходе работы автором работы были использованы различные игровые приложения для обучения детей. Исходя из полученного практического опыта, можно охарактеризовать некоторые приложения.

LightBot (Рис. 3.1.1)

Lightbot - это игра-головоломка по программированию, игра, которая использует игровую логику и которая прочно базируется на принципах программирования.

Просто давая роботу команды передвигаться и зажигать плитки, Lightbot позволяет игрокам наглядно на практике усвоить фундаментальные понятия программирования, такие как процедуры, и постепенно решая усложняющиеся уровни игры. [15]

Недостатки:

- Однообразные уровни, которые различаются только изменением позиции и количества плиток, которые нужно зажечь.
- Отсутствие циклов и условных операторов, что очень важно при обучении детей программированию.

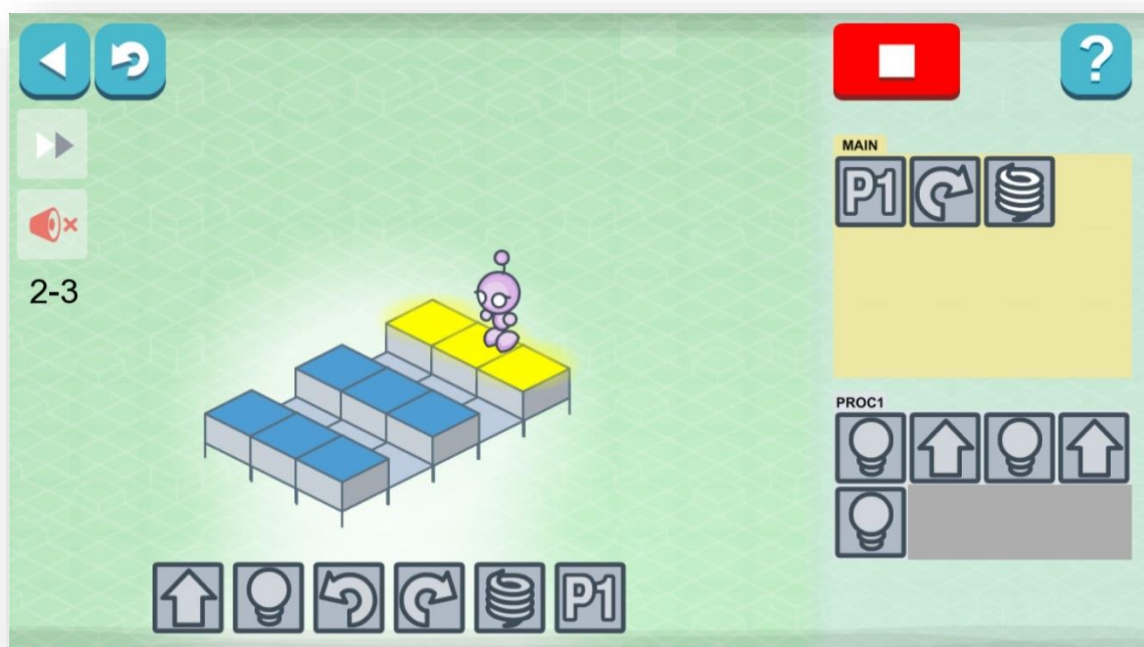


Рис. 3.1.1. LightBot

CodeCombat (Рис. 3.1.2)

CodeCombat - это платформа с открытым исходным кодом, которая позволяет научиться программировать на Python, играя в многопользовательскую игру. Платформа имеет большое количество персонажей, с которыми пользователь должен будет продвигаться по различным уровням, где можно столкнуться со сложными проблемами и противниками, для достижения целей каждого уровня вы должны использовать команды, типичные для языка программирования Python.

CodeCombat удастся естественным и ускоренным образом знакомить пользователей с языком программирования Python, поскольку игра способствует взаимодействию, открытию и обучению с помощью методов проб и ошибок. Со временем пользователь начинает осваивать навыки программирования, а также развиваются его логические мысли, что позволяет ему лучше анализировать любую проблему. [16]

Недостатки:

- Сложность. Слишком сложные уровни для детей, возраст которых менее 10 лет.
- Мало бесплатного контента – не более 10%, что не позволяет в достаточной мере усвоить механизм написания кода.

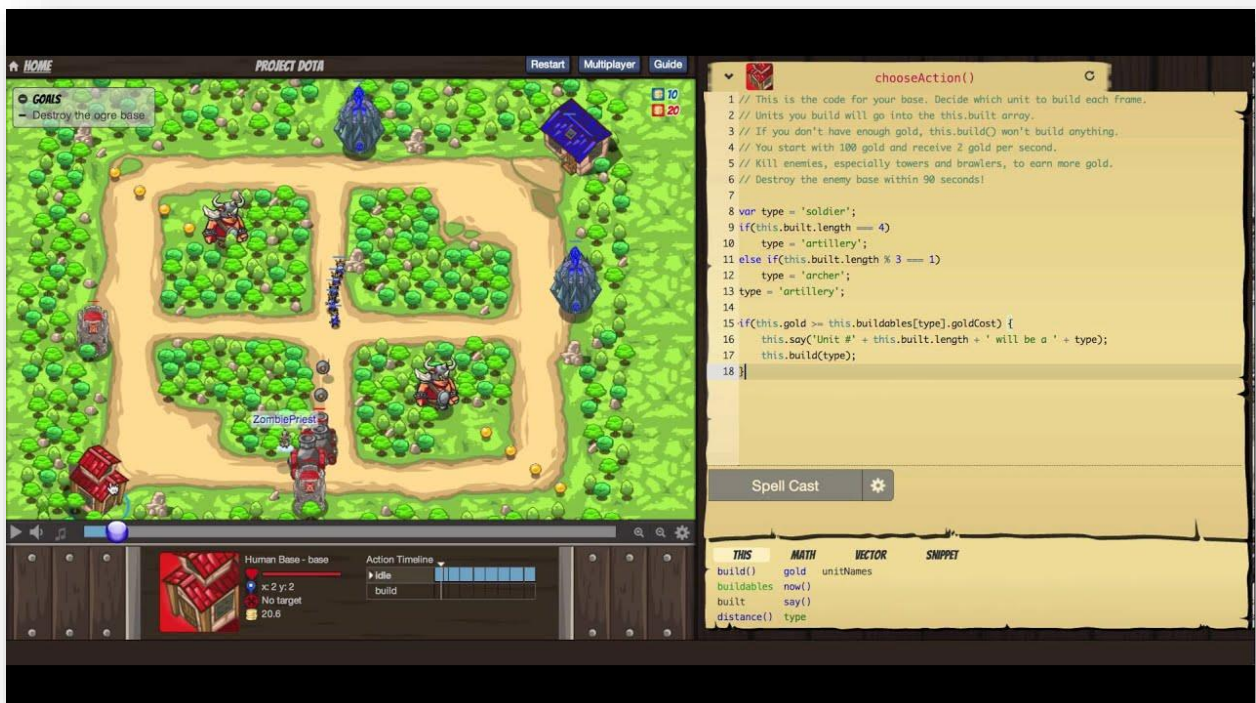


Рис. 3.1.2. CodeCombat

SpriteBox (Рис. 3.1.3)

SpriteBox - это простая, красочная и обучающая программированию детская игра для PC и мобильных платформ. Процесс обучения выполнен в виде головоломки с элементами платформера. Под управление игрокам дается выбрать персонажа и несколько десятков уровней. Прохождение локаций сопряжено с решением головоломок, основанных на синтаксисе Java.

Распространяется эта обучающая игра бесплатно. [17]

Недостатки:

- Отсутствие русского языка
- Нет возможности создать аккаунт, что не позволяет запомнить прогресс прохождения игры.

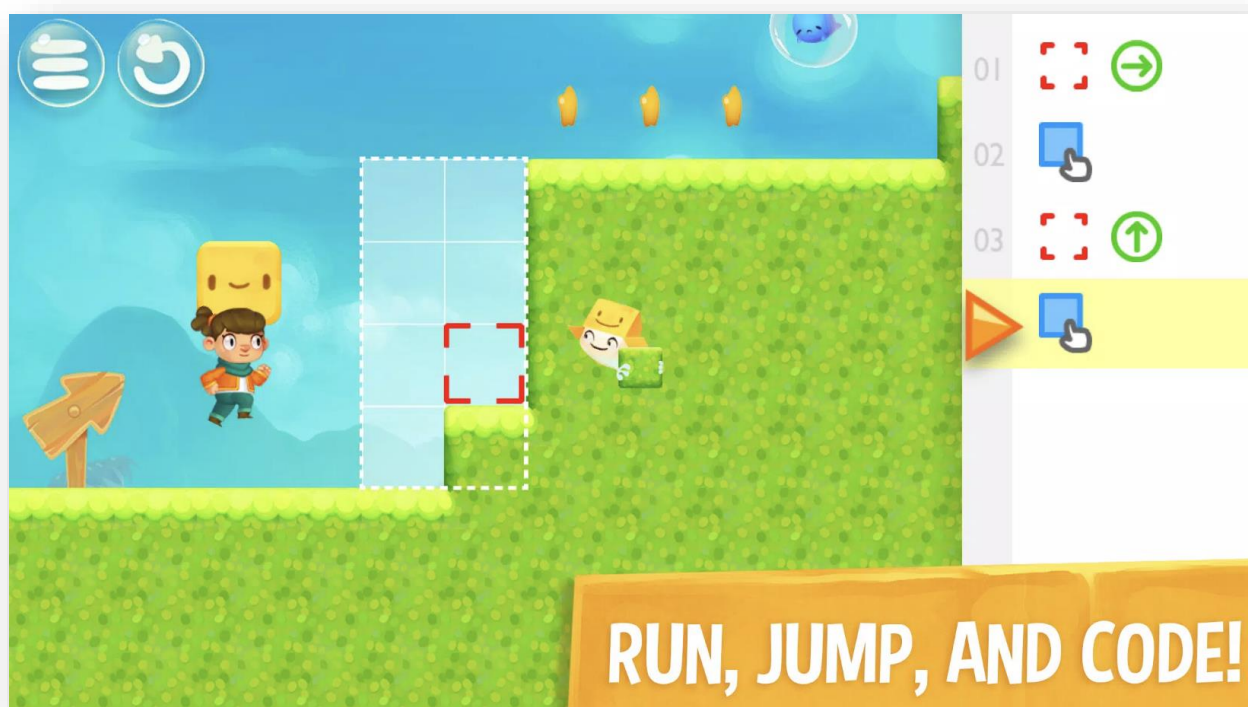


Рис. 3.1.3. SpriteBox

Сравнительная таблица описанных приложений

Проведение исследования рынка существующих приложений необходимо при подготовительной работе перед созданием собственного приложения. Сравнительный анализ положительных и отрицательных моментов аналогичных приложений позволяет с большей вероятностью сделать разрабатываемый продукт конкурентоспособным.

Исходя из анализа существующих игр, обучающих программированию, была составлена следующая сравнительная таблица (Таблица 3.1.4.).

Таблица 3.1.4. Сравнительная таблица

	LightBot	CodeCombat	SpriteBox	Программирование с Symon's Cat
Условные операторы	-	+	+	+
Циклы	-	+	+	+
Процедуры	+	+	-	+
Бесплатный контент	+	-	+	+
Возможность создать аккаунт	-	+	-	+
Родительский контроль	-	-	-	+
Разнообразные уровни	-	+	+	+
Русский язык	-	+	-	+

Приведенные существующие приложения, обучающие программированию, сравниваются с практической частью проекта. Исходя из таблицы, очевидно, что все проанализированные автором приложения имеют минусы. Проведенное исследование позволило сделать вывод, что спроектированное приложение является конкурентоспособным на рынке и имеет следующие преимущества перед конкурентами:

- Наличие условных операторов
- Наличие циклов
- Наличие процедур

- Бесплатный контент
- Возможность создать аккаунт
- Родительский контроль
- Разнообразные уровни
- Наличие русского языка

3.2 Диаграмма компонентов

Диаграмма компонентов - статическая структурная диаграмма, которая показывает разбиение программной системы на структурные компоненты и связи (зависимости) между компонентами.

Диаграмма компонентов описывает особенности физического представления системы. Диаграмма компонентов позволяет определить архитектуру разрабатываемой системы, установив зависимости между программными компонентами, в роли которых может выступать исходный, бинарный и исполняемый код.

Диаграмма компонентов разрабатывается для следующих целей:

- 1) визуализация общей структуры исходного кода программной системы;
- 2) спецификация исполнимого варианта программной системы;
- 3) обеспечение многократного использования отдельных фрагментов программного кода;
- 4) представление концептуальной и физической схем баз данных. [18]

На рисунке 3.2.1 представлена Component Диаграмма разрабатываемого приложения, на которой представлена структура приложения. Практическая часть реализована в форме «клиент-сервер» и состоит из 2 основных частей:

- Клиентской части
- Серверной части

В серверной части описаны:

- База данных, которая была реализована с помощью СУБД MySQL
- Hibernate классы для реализации связи ООП и реляционной базы данных.
- Логический модуль приложения

В клиентской части описаны:

- Веб интерфейсов
- Мобильных интерфейсов

- Client Interface Controller, позволяющий интегрировать информацию из серверной части в интерфейсы

Связь между клиентской и серверной части реализуется с помощью JS модуля и веб сервера.

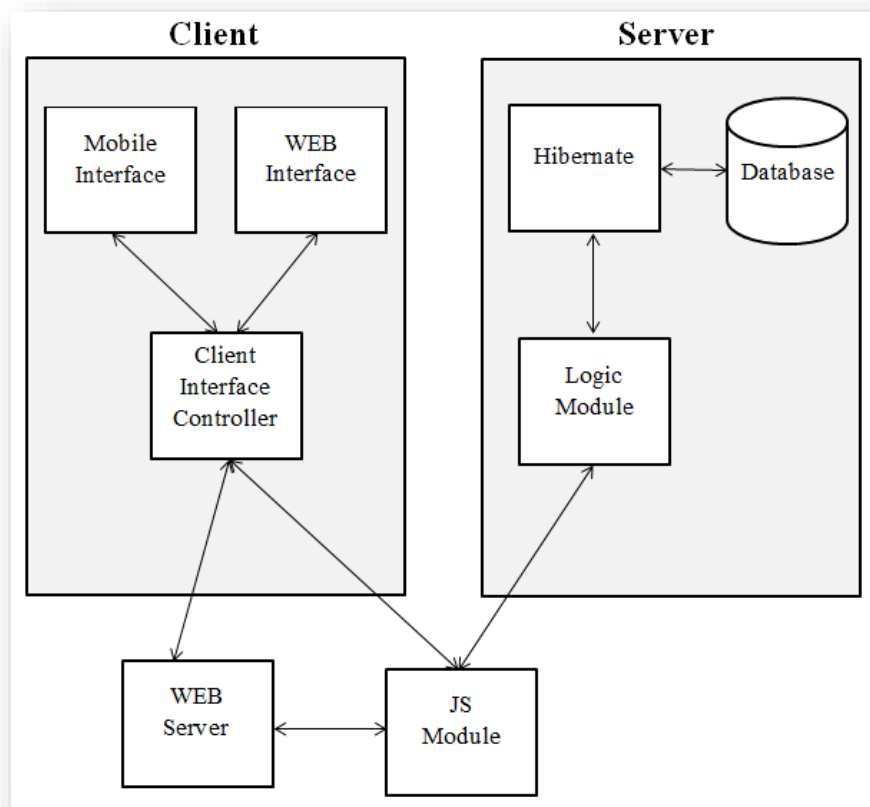


Рис. 3.2.1. Диаграмма компонентов

На основании вышеприведенной схемы логично предположить, что данная структура делает приложение более гибким (приложение легко изменяется и затрагивая ядро функциональности) и масштабируемым (благодаря использованной структуре приложение легко дополняется новыми функциями и интерфейсами, не затрагивая уже существующий код).

3.3 Диаграмма прецедентов использования

Диаграмма прецедентов использования - диаграмма, отражающая отношения между актерами (типами пользователей) и прецедентами (программными функциональностями) и являющаяся составной частью модели прецедентов, позволяющей описать систему на концептуальном уровне.

Прецедент - возможность моделируемой системы, благодаря которой пользователь может получить конкретный, измеримый и нужный ему результат. Прецедент

соответствует отдельному сервису системы, определяет один из вариантов её использования и описывает типичный способ взаимодействия пользователя с системой. Варианты использования обычно применяются для спецификации внешних требований к системе.

Основное назначение диаграммы - описание функциональности и поведения, позволяющее заказчику, конечному пользователю и разработчику совместно обсуждать проектируемую или существующую систему.

При моделировании системы с помощью диаграммы прецедентов необходимо:

- чётко отделить систему от её окружения;
- определить действующих лиц (актеров), их взаимодействие с системой и ожидаемую функциональность системы;
- определить в глоссарии предметной области понятия, относящиеся к детальному описанию функциональности системы (то есть прецедентов). [3, стр.29-34]

На рисунке 3.3.1 представлена диаграмма Use Case, на которой представлено 4 типа пользователей:

- 1) Гость - незарегистрированный пользователь
- 2) Зарегистрированный ребёнок
- 3) Зарегистрированный взрослый
- 4) Администратор

Возможности пользователей:

Гость:

- Регистрация
- Пройти Стартовый уровень

Зарегистрированный ребёнок:

- Авторизация
- Пройти Стартовый уровень
- Пройти уровни

Зарегистрированный взрослый:

- Авторизация
- Пройти Стартовый уровень
- Пройти уровни
- Посмотреть правильный сценарий прохождения уровня (подсказка)
- Просматривать свою статистику
- Просматривать статистику ребёнка

- Редактировать свою личную информацию
- Редактировать информацию о ребёнке
- Создать аккаунт для ребёнка

Админ:

- Авторизация
- Пройти Стартовый уровень
- Пройти уровни
- Посмотреть правильный сценарий прохождения уровня (подсказка)
- Просматривать свою статистику
- Просматривать статистику ребёнка
- Редактировать свою личную информацию
- Редактировать информацию о ребёнке
- Создать аккаунт для ребёнка
- Редактировать пользовательские таблицы (добавить, изменить, удалить)
- Редактировать таблицы уровней (добавить, изменить, удалить)

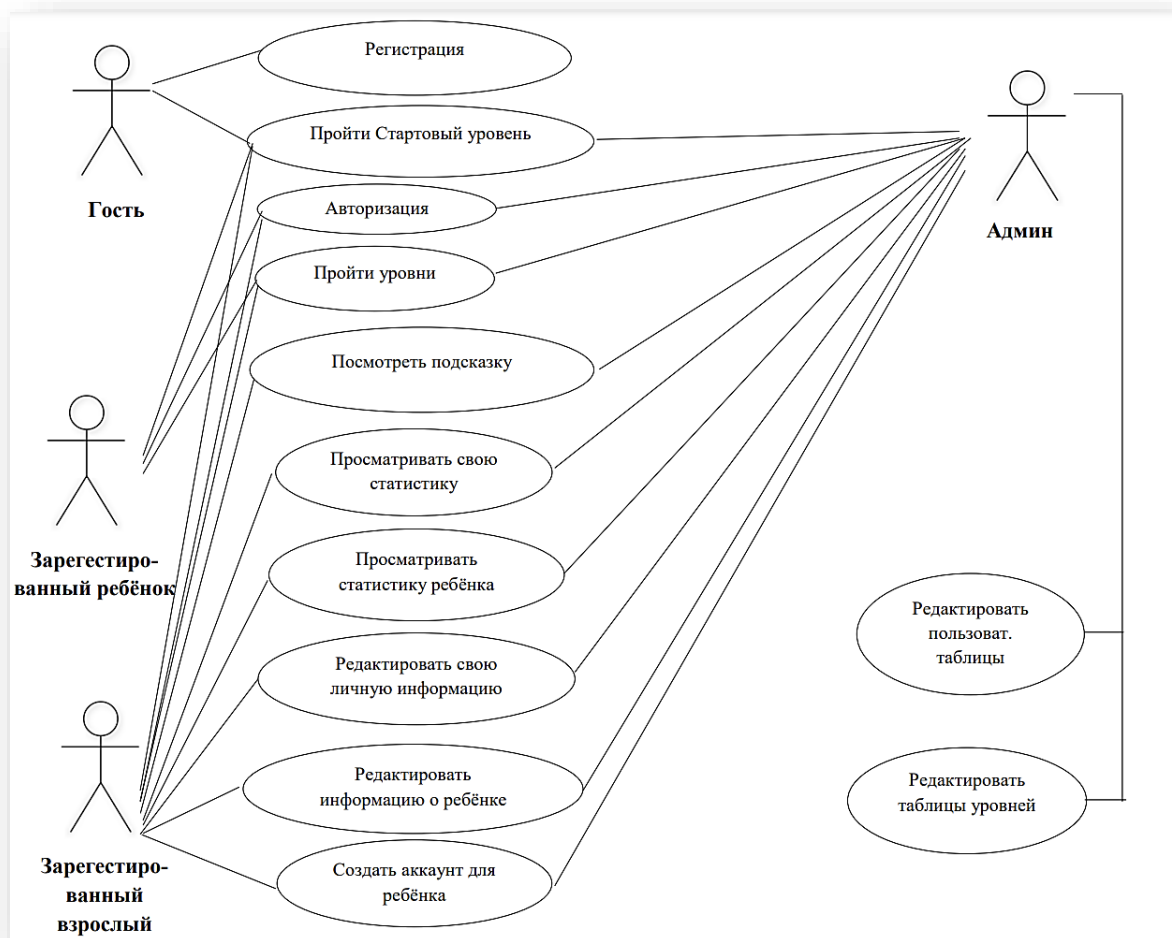


Рис. 3.3.1. Диаграмма прецедентов использования

Таким образом, можно сделать вывод, что спроектированное приложение имеет следующую уникальную особенность: в приложении присутствуют 2 типа зарегистрированных пользователей: ребенок и взрослый. Только взрослый может зарегистрировать своего ребенка и имеет больше прав, например, посмотреть ответы, редактировать свою информацию и ребенка и просматривать статистику. Данное разделение было сделано для того, чтобы взрослый мог контролировать и направлять процесс обучения ребенка программированию, что существенно увеличивает скорость и эффективность усвоения материала.

3.4 ER Диаграмма

Диаграмма отношений сущностей (ERD) - это визуальное представление базы данных, которое показывает, как связаны таблицы внутри базы данных. Диаграмма ER состоит из двух типов объектов - сущностей и отношений. Сущность в этом контексте - это компонент данных из набора данных, отображаемый в виде фигуры на холсте. Отношения между сущностями представлены в виде строк, которые имеют специальные окончания строк, называемые кардинальностями, которые описывают, как два элемента базы данных взаимодействуют друг с другом. [19]

Реляционная структурная модель базы данных соответствует следующим требованиям:

- 1) Отношения находятся в нормальной форме
- 2) Ограничения структуры (уникальность ключа)
- 3) Реляционные операторы

Этапы создания базы данных:

- 1) Сбор необходимой информации
- 2) Создание проекта
- 3) Физическая реализация

Схема базы данных включает в себя следующие таблицы (Рис.3.4.1):

- 1) USER – информация о зарегистрированном пользователе; включает в себя следующие атрибуты: user_id (идентификатор пользователя), user_login (логин пользователя), user_password (пароль пользователя), user_name (имя пользователя), user_date (дата рождения пользователя), user_status (статус пользователя: ребёнок или родитель).

2) PARENT_CHILD – таблица, показывающая связь между родителями и их детьми; включает в себя следующие атрибуты: connection_id (идентификатор связи), parent_id (идентификатор родителя), child_id (идентификатор ребёнка).

3) BLOCK – блок уровней; включает в себя следующие атрибуты: block_id (идентификатор блока), block_name (название блока).

4) LEVEL – информация об уровне; включает в себя следующие атрибуты: level_id (идентификатор уровня), description (описание уровня), solution (решение), block_id (идентификатор блока).

5) SESSION – информация о сессиях; включает в себя следующие атрибуты: session_id (идентификатор сессии), time_start (время начала сессии), time_end (время окончания сессии), user_id (идентификатор пользователя), level_id (идентификатор уровня).

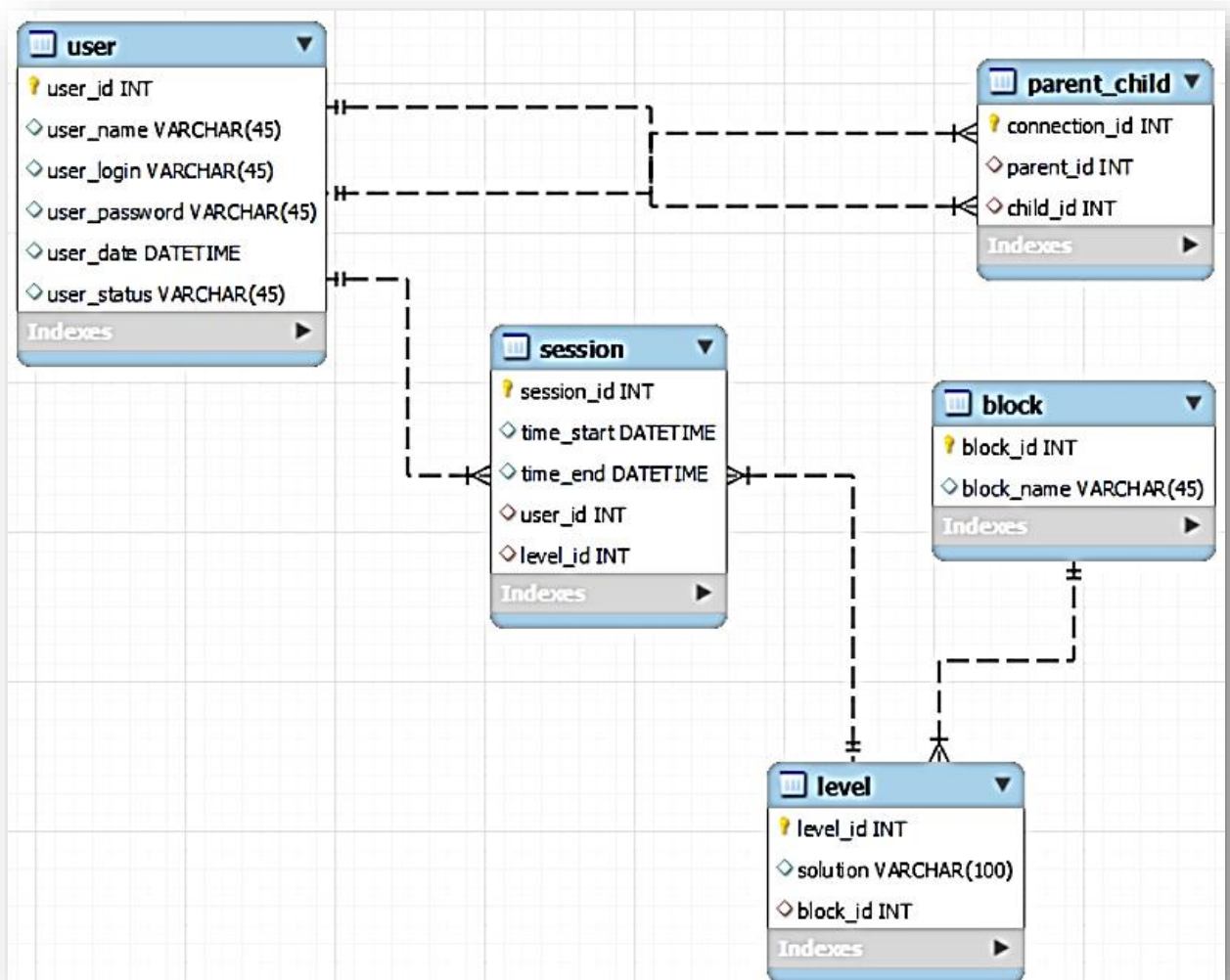


Рис. 3.4.1. ER Диаграмма

Представленная база данных была реализована в СУБД MySQL, которая была описана выше. Таблица базы данных находится в 3 нормальной форме.

3.5 Диаграммы последовательностей

Диаграмма последовательности (англ. sequence diagram) - UML-диаграмма, на которой для некоторого набора объектов на единой временной оси показан жизненный цикл объекта (создание-деятельность-уничтожение некой сущности) и взаимодействие актеров (действующих лиц) информационной системы в рамках прецедента.

Основными элементами диаграммы последовательности являются обозначения объектов (прямоугольники с названиями объектов), вертикальные «линии жизни», отображающие течение времени, прямоугольники, отражающие деятельность объекта или исполнение им определенной функции (прямоугольники на пунктирной «линии жизни»), и стрелки, показывающие обмен сигналами или сообщениями между объектами. [4, стр.41-45]

На рисунке 3.5.1 представлена диаграмма последовательности одного из прецедентов использования разработанного приложения (Авторизация). Актер вводит логин и пароль через веб интерфейс. Данные передаются логическому модулю, который передает их фреймворку Hibernate. Hibernate передает логин, пароль и сессию в базу данных. Если аккаунт есть в базе данных, то логическому модулю передается идентификатор пользователя, и далее следует переход на интерфейс профиля. Если аккаунта не существует, то логическому модулю передается сообщение об ошибке, и далее следует переход на интерфейс с сообщением об ошибке.

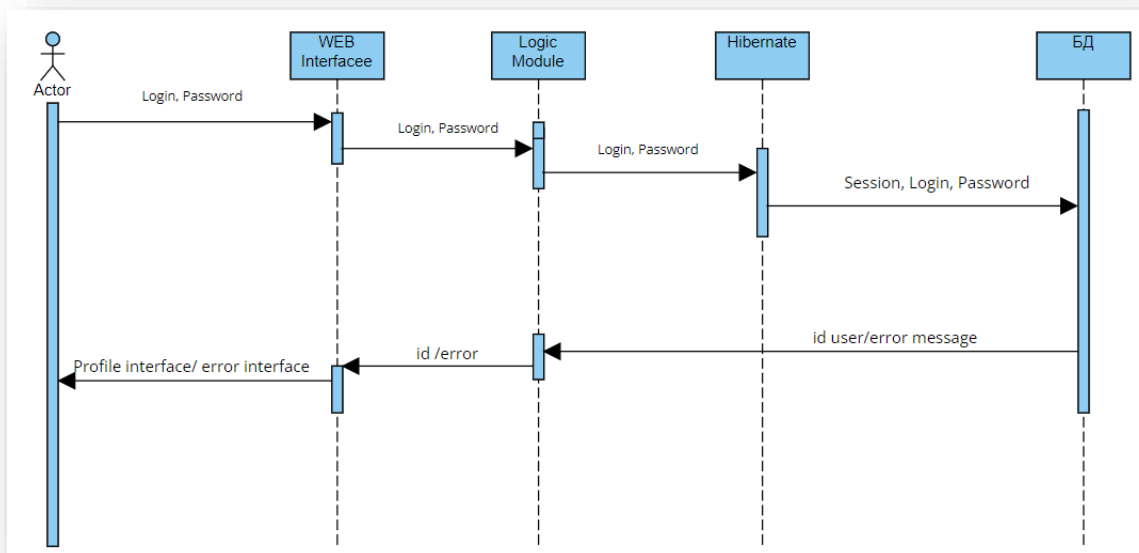


Рис. 3.5.1. Диаграмма последовательности «Авторизация»

На рисунке 3.5.2 представлена диаграмма последовательности одного из прецедентов использования разработанного приложения (Прохождение уровней). Актер использует команды через веб интерфейс. Данные команды передаются JS модулю. Модуль

проверяет, является ли последовательность команд верной для прохождения уровня. Если пользователь использовал правильный ход, то отображается модальное окно с победой, в противном случае – с проигрышем.

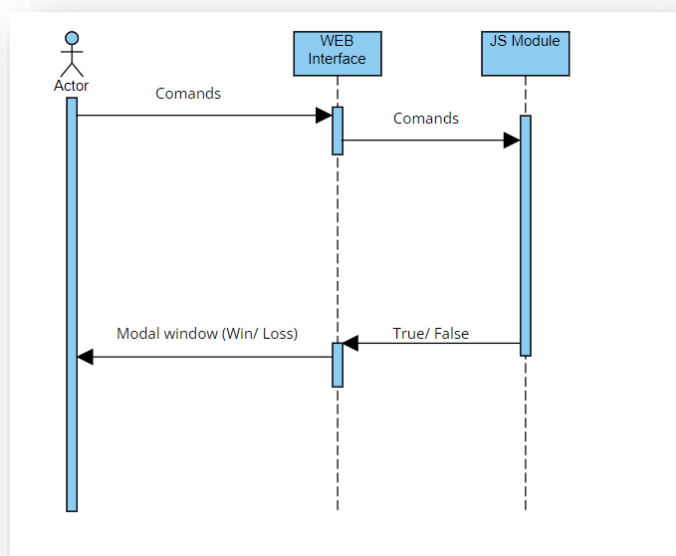


Рис. 3.5.2. Диаграмма последовательности «Прохождение уровня»

3.6 Дерево интерфейсов

На рисунке 3.6.1 представлено дерево интерфейсов приложения, на котором представлены 12 окон интерфейсов:

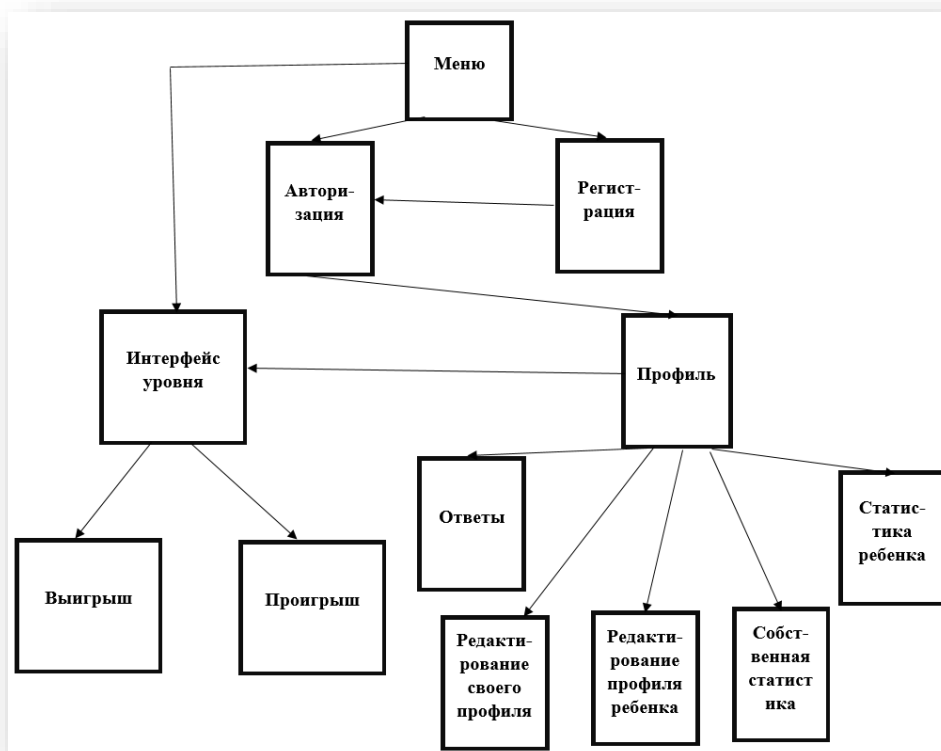


Рис.3.6.1. Дерево интерфейсов

Интерфейс «Меню» (Рис. 3.6.2) содержит 3 кнопки:

- «Зайди», при нажатии на которую следует переход на интерфейс авторизации
- «Зарегистрируйся», при нажатии на которую следует переход на интерфейс регистрации
- «Играй», при нажатии на которую следует переход на интерфейс уровня



Рис.3.6.2. Интерфейс «Меню»

Интерфейс «Авторизация» (Рис. 3.6.3) содержит форму аутентификации с 2 полями: логин и пароль. При успешной аутентификации следует переход на интерфейс профиля. При неудачной аутентификации появится сообщение об ошибке. При нажатии на кнопку «Назад» следует переход на интерфейс «Меню».

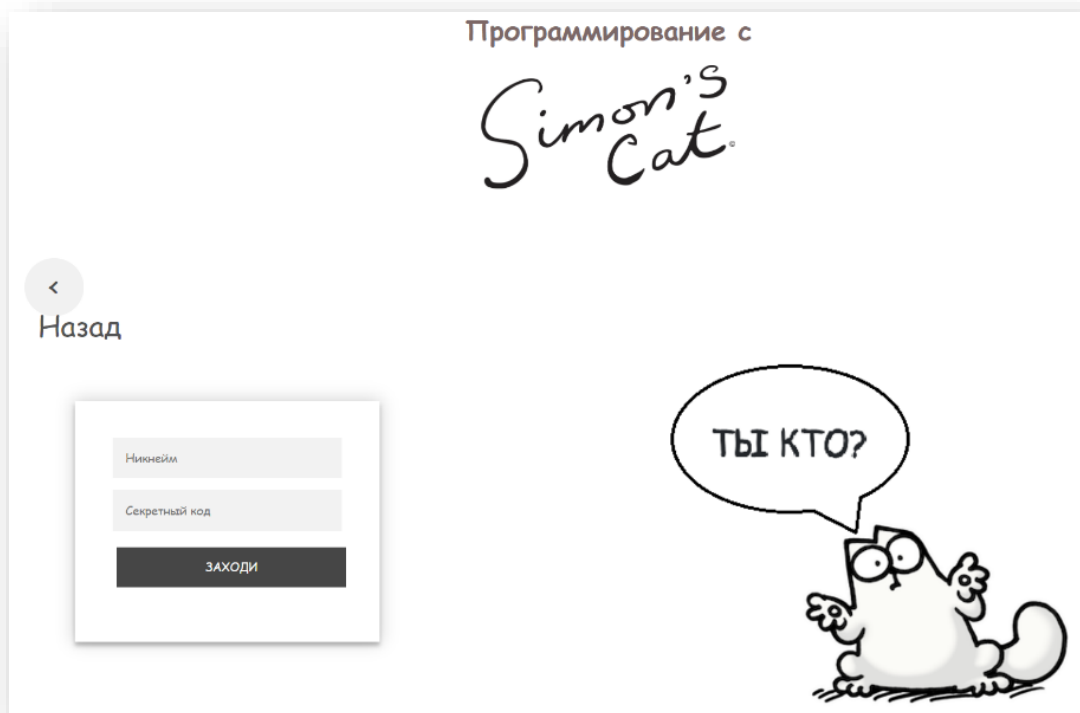


Рис.3.6.3. Интерфейс «Авторизация»

Интерфейс «Регистрация» (Рис. 3.6.4) содержит форму регистрации с 5 полями: имя, логин, дата рождения, пароль и подтверждение пароля. При успешной регистрации следует переход на интерфейс «Меню». При неудачной регистрации появится сообщение об ошибке. При нажатии на кнопку «Назад» следует переход на интерфейс «Меню».

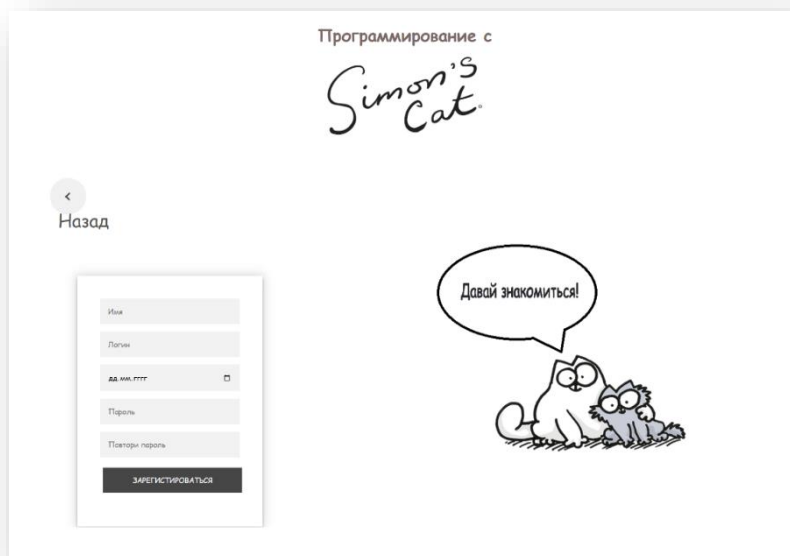


Рис.3.6.4. Интерфейс «Регистрация»

Интерфейс «Профиль» (Рис. 3.6.5) содержит информацию о пользователе и следующие кнопки:

- «Играй», при нажатии на которую следует переход на интерфейс уровня;
- «Редактировать информацию», при нажатии на которую следует переход на интерфейс «Редактирование своего профиля»;
- «Ответы», при нажатии на которую следует переход на интерфейс «Ответы»;
- «Статистика», при нажатии на которую следует переход на интерфейс «Собственная статистика»;
- «Статистика ребенка», при нажатии на которую следует переход на интерфейс «Статистика ребенка»;
- «Редактировать информацию о ребенке», при нажатии на которую следует переход на интерфейс «Редактирование профиля ребенка»;
- «Создать аккаунт для ребенка»

При нажатии на кнопку «Назад» следует переход на интерфейс «Меню».



Рис.3.6.5. Интерфейс «Профиль»

Интерфейс «Ответы» (Рис. 3.6.6) содержит изображения, нажав на которые, можно посмотреть прохождение уровней. При нажатии на кнопку «Назад» следует переход на интерфейс «Профиль».



Рис.3.6.6. Интерфейс «Ответы»

Интерфейс уровня (Рис. 3.7.7) включает в себя 4 интерфейса уровня и содержит игровую зону, где пользователь следит за прохождением игры, команды, которые пользователь может использовать и код, где пользователь использует необходимые команды. При успешном прохождении уровня следует переход на интерфейс «Выигрыш». Если пользователь не прошел уровень, следует переход на интерфейс «Проигрыш». При нажатии на кнопку «Назад» следует переход на интерфейс «Профиль».

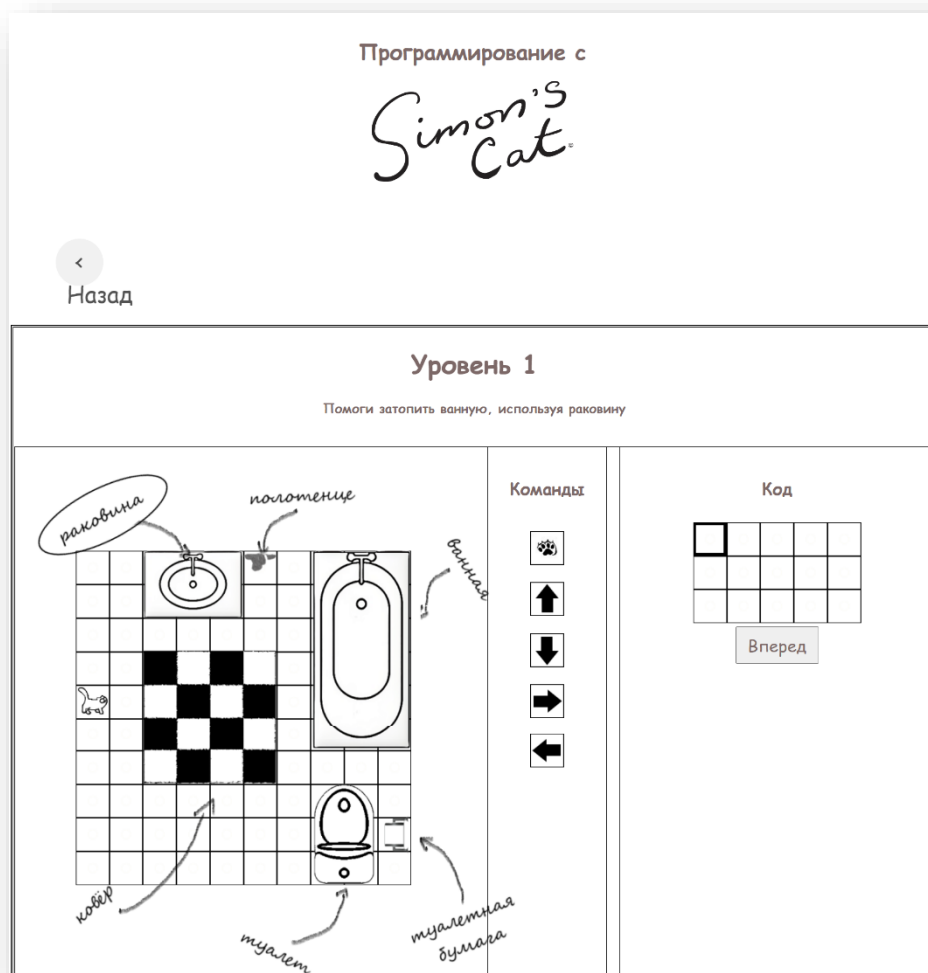


Рис.3.7.7. Интерфейс уровня

Интерфейс «Выигрыш» (Рис. 3.7.8) является модальным окном и включает в себя уникальное видео, которое меняется в зависимости от уровня, и 2 кнопки: «Повторить» и «Следующий уровень». При нажатии на кнопку «Повторить» следует переход на интерфейс предыдущего уровня для повторного прохождения. При нажатии на кнопку «Следующий уровень» следует переход на интерфейс следующего уровня.



Рис.3.7.8. Интерфейс «Выигрыш»

Интерфейс «Проигрыш» (Рис. 3.7.9) является модальным окном и включает в себя изображение и кнопку «Попробуй еще раз». При нажатии на данную кнопку следует переход на интерфейс предыдущего уровня для повторного прохождения.



Рис.3.7.9. Интерфейс «Проигрыш»

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломной работы было создано приложение, обучающее детей программированию в легкой интуитивной игровой форме.

Приложение структурировано так, что может быть расширено следующим образом:

- Может быть спроектирован и добавлен интерфейс под мобильные операционные системы Android и IOS.
- Могут быть добавлены новые типы пользователей.
- Может быть добавлено неограниченное количество уровней

Приложение является гибким и масштабируемым, так как, не изменяя ядра, может быть увеличено и может переноситься с одной платформы на другую без существенных изменений.

Все сформулированные теоретические цели работы были реализованы на 100%, то есть:

- Было проведено исследование существующих ORM технологий и их сравнительный анализ.
- Было реализован сравнительный анализ существующих фреймворков для языка программирования JS
- Были детально изучены фреймворки React, Hibernate и Spring и интеграция их в проект
- Была исследована структуры образовательных приложений
- Был проведен сравнительный анализ существующих на рынке образовательных приложений

Все поставленные практические задачи были выполнены на 100%, то есть:

- Был создан проект собственного образовательного приложения, включающий все изученные технологии.
- Было реализовано приложение на Java и JS
- Было осуществлено тестирование приложения вместе с учениками академии “IT Step”

В качестве дальнейшего развития проекта планируется:

- Спроектировать интерфейс под мобильные операционные системы Android и IOS.
- Добавить возможность отслеживать статистику

- Добавить возможность пройти уровень с помощью существующих языков программирования
- Добавить возможность писать команды (вместо нажатия на кнопки) для прохождения уровня
- Добавить уровни с более высокой сложностью для подростков

Вышеуказанные улучшения проекта автором планируется реализовать в научной работе на Мастерате.

БИБЛИОГРАФИЯ

Бумажные источники

1. Разумова Т. «Компьютерный яд». Наука и жизнь. № 6, 2012
2. Козьмина Ю, Харроп Р, Шефер К, Хо К «Spring 5 для профессионалов, пятое издание», 2019
3. Бабич А.В. «Введение в UML», 2-е издание исправленное, 2016
4. Комолова Н., Яковлева Е. «Adobe Photoshop СС для всех», 2014

Интернет-источники

5. «ORM object-relational mapping (объектно-реляционное отображение) — Варианты реализации (Active Record и Data Mapper) и альтернативы», <https://intellect.icu/orm-object-relational-mapping-obektno-relyatsionnoe-otobrazhenie-varianty-realizatsii-active-record-i-data-mapper-i-alternativy-4701> (03.06.2022)
6. «Наиболее популярные Javascript фреймворки для быстрой веб разработки: что выбрать?», Николай Стариков, 19.06.2019, <https://stfalcon.com/ru/blog/post/javascript-frameworks-for-software-development#:~:text=%D0%93%D0%BE%D0%B2%D0%BE%D1%80%D1%8F%20%D0%BF%D1%80%D0%BE%D1%81%D1%82%D1%8B%D0%BC%20%D1%8F%D0%B7%D1%8B%D0%BA%D0%BE%D0%BC%2C%20Javascript%20%D1%84%D1%80%D0%B5%D0%B9%D0%BC%D0%B2%D0%BE%D1%80%D0%BA,%D0%B2%D0%B5%D0%B1%2D%D1%81%D1%82%D1%80%D0%B0%D0%BD%D0%B8%D1%86%D0%B5%20%D0%B8%D0%BB%D0%B8%20%D0%B2%20%D0%BF%D1%80%D0%B8%D0%BB%D0%BE%D0%B6%D0%B5%D0%BD%D0%B8%D0%B8.> (03.06.2022)
7. «React JS», <https://bizzapps.ru/p/react-js/> (03.06.2022)
8. «Плюсы и минусы React: виртуальная DOM, синтаксис JSX и другие аргументы для спора», 14.11.2021, <https://nuancesprog.ru/p/14500/> (03.06.2022)
9. «Руководство по Hibernate. Архитектура.», <https://proselyte.net/tutorials/hibernate-tutorial/architecture/> (03.06.2022)
10. «Ответы на вопросы на собеседование Object relational mapping (ORM), Hibernate (часть 1)», <https://jsehelper.blogspot.com/2016/01/object-relational-mapping-orm-hibernate.html> (03.06.2022)
11. «Зачем использовать Spring?», <https://ru.stackoverflow.com/questions/428719/%D0%97%D0%B0%D1%87%D0%B5%D0%BC->

- [%D0%B8%D1%81%D0%BF%D0%BE%D0%BB%D1%8C%D0%B7%D0%BE%D0%B2%D0%B0%D1%82%D1%8C-spring](#) (03.06.2022)
12. «Система управления базами данных MySQL»,
https://depix.ru/articles/sistema_upravleniya_bazami_dannyh_mysql (03.06.2022)
 13. «JetBrains IntelliJ IDEA», <https://itpro.ua/product/jetbrains-intellij-idea/?tab=description> (03.06.2022)
 14. «Почему IntelliJ IDEA Продуктивная разработка на Java»,
<https://www.jetbrains.com/ru-ru/idea/> (03.06.2022)
 15. «LIGHTBOT», <http://robot.nios.ru/useful/167> (03.06.2022)
 16. «Как научиться программировать на Python, играя с CodeCombat»,
<https://blog.desdelinux.net/ru/como-aprender-programar-python-juegas-codecombat/> (03.06.2022)
 17. «SpriteBox Coding», <https://tabsgame.ru/4523-spritebox-coding.html> (03.06.2022)
 18. «Диаграмма компонентов (component diagram)»,
http://imlearning.ru/netcat_files/file/FSIS/%D0%A4%D0%A1%D0%98%D0%A1_%D1%81%D0%B5%D0%BC%D0%B8%D0%BD%D0%B0%D1%80-9_%D0%94%D0%B8%D0%B0%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D0%B0-%D0%BA%D0%BE%D0%BC%D0%BF%D0%BE%D0%BD%D0%B5%D0%BD%D1%82%D0%BE%D0%B2.pdf (03.06.2022)
 19. «ER-диаграммы», [https://lucidchart.zendesk.com/hc/ru/articles/207299756-ER-%D0%B4%D0%B8%D0%B0%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D1%8B#:~:text=%D0%94%D0%B8%D0%B0%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D0%B0%20%D0%BE%D1%82%D0%BD%D0%BE%D1%88%D0%B5%D0%BD%D0%B8%D0%B9%20%D1%81%D1%83%D1%89%D0%BD%D0%BE%D1%81%D1%82%D0%B5%D0%B9%20\(ERD\)%20%E2%80%94,%D0%B2%20%D0%B2%D0%B8%D0%B4%D0%B5%20%D1%84%D0%B8%D0%B3%D1%83%D1%80%D1%8B%20%D0%BD%D0%B0%20%D1%85%D0%BE%D0%BB%D1%81%D1%82%D0%B5](https://lucidchart.zendesk.com/hc/ru/articles/207299756-ER-%D0%B4%D0%B8%D0%B0%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D1%8B#:~:text=%D0%94%D0%B8%D0%B0%D0%B3%D1%80%D0%B0%D0%BC%D0%BC%D0%B0%20%D0%BE%D1%82%D0%BD%D0%BE%D1%88%D0%B5%D0%BD%D0%B8%D0%B9%20%D1%81%D1%83%D1%89%D0%BD%D0%BE%D1%81%D1%82%D0%B5%D0%B9%20(ERD)%20%E2%80%94,%D0%B2%20%D0%B2%D0%B8%D0%B4%D0%B5%20%D1%84%D0%B8%D0%B3%D1%83%D1%80%D1%8B%20%D0%BD%D0%B0%20%D1%85%D0%BE%D0%BB%D1%81%D1%82%D0%B5) (03.06.2022)
 20. «Movavi Video Editor»,
https://www.mssoft.ru/Makers/Movavi/Movavi_Video_Editor_10/ (03.06.2022)
 21. «Основы Hibernate», Гобозов Георгий, <https://habr.com/ru/post/29694/> (03.06.2022)

ПРИЛОЖЕНИЕ

1level.jsp – пример создания страницы для уровня

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
<%@ page import = "java.io.*, java.util.*" %>
<html lang="en">
<head>
  <title>Программирование с Symon's Cat</title>
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta charset = "utf-8" />
  <link rel="shortcut icon" href="images/icon.png" type="image/png">
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>

<div class="head">
  <h1>Программирование с </h1>
  

</div>
<div class="b">
  <a href="index.jsp" class="previous round"   valign="top"> <br> Назад</a>
</div>

<div class="container1" width="100%">

  <div class="title">
    <h1>Уровень 1 </h1>
    <h4>Помоги затопить ванную комнату, используя раковину</h4>
  </div>

  <div class="game" >
    <div class='world2'></div>
    <div class='world_arrows'></div>
    <div id='world' class="world"></div>
```

</div>

<div class="comands">

<h4>Команды</h4>

<button id="lapka"><div class='lapka'></div></button>

<button id="up"><div class='up'> </div></button>

<button id="down"><div class='down'> </div></button>

<button id="right"><div class='right'> </div> </button>

<button id="left"><div class='left'> </div></button>

</div>

<div class="coding">

<h4>Код</h4>

<div id='world_coding'>

</div>

<button valign="top" id="go" class="gamecursor"
onclick="go(1)">Вперед</button></p>

</div>

</div>

<div id="myModal" class="modal">

<div class="modal-content" align="center">

<video class="video" id="video" width="70%" loop>

<source src="images/video/victory.mp4" type="video/mp4">

</video>

<button onclick="**location**.reload(); return false;"> Повторить</button>

<button onclick="**window**.location.href =
'http://localhost:8080/login_reg_war_exploded/2level.jsp';">Следующий уровень</button>

</div>

</div>

<div id="myModal2" class="modal">

<div class="modal-content" align="center">


```
<br>
<button valign="top" onclick="location.reload(); return false;"> Попробуй еще
раз</button></p>
</div>
```

```
</div>
```

```
</body>
```

```
<script src="game.js"></script>
```

```
</html>
```

game.js - JS файл, где прописана логика игры

```
cat = {
  x: 0,
  y: 4
} //местоположение кота
```

```
cursor = {
  i: 0,
  j: 0
}
```

```
//0 - полотенце
```

```
//1 - раковина
```

```
//7 - препятствия
```

```
//6 - туалетная бумага
```

```
map = [
  [2,2,1,1,1,7,2,7,7,7],
  [2,2,1,1,1,2,2,7,7,7],
  [2,2,2,2,2,2,2,7,7,7],
```

```

[2,2,7,7,7,7,2,7,7,7],
[5,2,7,7,7,7,2,7,7,7],
[2,2,7,7,7,7,2,7,7,7],
[2,2,7,7,7,7,2,2,2,2],
[2,2,2,2,2,2,2,7,7,2],
[2,2,2,2,2,2,2,7,6],
[2,2,2,2,2,2,2,7,2]
] //массив карты, где передвигается персонаж

arr = [0,1,7,6]

map2 = [
  [1,2,2,2,2],
  [2,2,2,2,2],
  [2,2,2,2,2]
] //массив кода

var el = document.getElementById('world');
var el2 = document.getElementById('world_coding');

function drawWorld() { //функция рисования игровой карты и карты команд
  el.innerHTML = "";
  el2.innerHTML = "";

  for (var y = 0; y < map.length; y = y + 1) { //игровая зона
    for (var x = 0; x < map[y].length; x = x + 1) {
      if (map[y][x] === 0) {
        el.innerHTML += "<div class='cell'></div>";
      } else if (map[y][x] === 1) {
        el.innerHTML += "<div class='cell'></div>";
      } else if (map[y][x] === 2) {
        el.innerHTML += "<div class='cell'></div>";
      } else if (map[y][x] === 3) {

```

```

    el.innerHTML += "<div class='ground'></div>";
  } else if (map[y][x] === 4) {
    el.innerHTML += "<div class='cell'></div>";
  } else if (map[y][x] === 5) {
    el.innerHTML += "<div class='cat'></div>";
  } else if (map[y][x] === 6) {
    el.innerHTML += "<div class=' cell '></div>";
  } else if (map[y][x] === 7) {
    el.innerHTML += "<div class='cell'></div>";
  }

}
el.innerHTML += "<br>";
}

```

```

for (var i = 0; i < map2.length; i = i + 1) { //команды
  for (var j = 0; j < map2[i].length; j = j + 1) {

```

```

    if (map2[i][j] === 1) {
      el2.innerHTML += "<div class='cursor'></div>";
    } else if (map2[i][j] === 2) {
      el2.innerHTML += "<div class='coin'></div>";
    } else if (map2[i][j] === 3) {
      el2.innerHTML += "<div class='right'></div>";
    } else if (map2[i][j] === 4) {
      el2.innerHTML += "<div class='left'></div>";
    } else if (map2[i][j] === 5) {
      el2.innerHTML += "<div class='up'></div>";
    } else if (map2[i][j] === 6) {
      el2.innerHTML += "<div class='down'></div>";
    } else if (map2[i][j] === 7) {
      el2.innerHTML += "<br>";
    } else if (map2[i][j] === 8) {
      el2.innerHTML += "<div class='lapka'></div>";
    }
  }
}

```

```

    }

    }
    el2.innerHTML += "<br>";
}
}

drawWorld();
document.getElementById("right").onclick = c_right1;
document.getElementById("left").onclick = c_left;
document.getElementById("up").onclick = c_up;
document.getElementById("down").onclick = c_down;
document.getElementById("lapka").onclick = c_lapka;

function c_right1() { //функция передвижения кота вправо
    map2[cursor.i][cursor.j] = 3;
    cursor.j = cursor.j + 1;
    if(cursor.j ===5 || cursor.j ===11) {cursor.i++; cursor.j=0}
    map2[cursor.i][cursor.j] = 1;
    drawWorld();
}

function c_left() { //функция передвижения кота влево
    map2[cursor.i][cursor.j] = 4;
    cursor.j = cursor.j + 1;
    if(cursor.j ===5 || cursor.j ===11) {cursor.i++; cursor.j=0}
    map2[cursor.i][cursor.j] = 1;
    drawWorld();
}

function c_up() { //функция передвижения кота вверх
    map2[cursor.i][cursor.j] = 5;

```

```

    cursor.j = cursor.j + 1;
    if(cursor.j ===5 || cursor.j ===11) {cursor.i++; cursor.j=0}
    map2[cursor.i][cursor.j] = 1;
    drawWorld();

}

```

```

function c_down(){ //функция передвижения кота вниз
    map2[cursor.i][cursor.j] = 6;
    cursor.j = cursor.j + 1;
    if(cursor.j ===5 || cursor.j ===11) {cursor.i++; cursor.j=0}
    map2[cursor.i][cursor.j] = 1;
    drawWorld();
}

```

```

function c_lapka(){ //функция лапки

    map2[cursor.i][cursor.j] = 8;
    cursor.j = cursor.j + 1;
    if(cursor.j ===5 || cursor.j ===11) {cursor.i++; cursor.j=0}
    map2[cursor.i][cursor.j] = 1;
    drawWorld();
}

```

```

var modal = document.getElementById("myModal");
var modal2 = document.getElementById("myModal2");
var btn = document.getElementById("myBtn");
var span = document.getElementsByClassName("close")[0];
video = document.getElementById("video");

```

```

function sleep(ms) {
    return new Promise(resolve => setTimeout(resolve, ms));
}

```

```

async function go(n){

for (var i = 0; i < map2.length; i = i + 1) {
  for (var j = 0; j < map2[i].length; j = j + 1) {
    if(map2[i][j]===3) {
      if(contains(arr,map[cat.y][cat.x+1])) {modal2.style.display = "block"; break;}
      map[cat.y][cat.x] = 3;
      if(map[cat.y][cat.x] = 3)
        cat.x = cat.x + 1;
      map[cat.y][cat.x] = 5;
      drawWorld();
      await sleep(1000);
    }
    else if(map2[i][j]===4) {
      if(contains(arr, map[cat.y][cat.x-1])) {modal2.style.display = "block"; break;}
      map[cat.y][cat.x] = 3;
      cat.x = cat.x - 1;
      map[cat.y][cat.x] = 5;
      drawWorld();
      await sleep(1000);
    }
    else if(map2[i][j]===5)
    {
      if(contains(arr,map[cat.y-1][cat.x])) {modal2.style.display = "block"; break;}
      map[cat.y][cat.x] = 3;
      cat.y = cat.y - 1;
      map[cat.y][cat.x] = 5;
      drawWorld();
      await sleep(1000);
    }
    else if(map2[i][j]===6)
    {
      if(contains(arr, map[cat.y+1][cat.x])) {modal2.style.display = "block"; break;}
      map[cat.y][cat.x] = 3;

```

```

        cat.y = cat.y + 1;
        map[cat.y][cat.x] = 5;
        drawWorld();
        await sleep(1000);
    }
}
}

```

```

    if(map[cat.y][cat.x+1]===n || map[cat.y][cat.x-1]===n || map[cat.y+1][cat.x]===n ||
map[cat.y-1][cat.x]===n)
    {
        modal.style.display = "block";
        video.play();

    }
    else modal2.style.display = "block";

}

```

```

span.onclick = function() {
    modal.style.display = "none";
    modal2.style.display = "none";
    video.pause();
}

```

```

window.onclick = function(event) {
    if (event.target == modal) {
        modal.style.display = "none";
        video.pause();
    }
}

```

```

    if (event.target == modal2) {
        modal2.style.display = "none";
    }
}

```

```

    }
}

function contains(arr, elem) {
    for (var i = 0; i < arr.length; i++) {
        if (arr[i] === elem) {
            return true;
        }
    }
    return false;
}

```

UserEntity.java – пример создания класса пользователя исходя из базы данных

```
package entity;
```

```
import javax.persistence.*;
import java.sql.Timestamp;
```

```

@Entity
@Table(name = "user", schema = "symonscat", catalog = "")
public class UserEntity {
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Id
    @Column(name = "user_id")
    private int userId;
    @Basic
    @Column(name = "user_name")
    private String userName;
    @Basic
    @Column(name = "user_login")
    private String userLogin;
    @Basic
    @Column(name = "user_password")
    private String userPassword;
    @Basic

```



```
@Column(name = "user_date")
private Timestamp userDate;

@Basic
@Column(name = "user_status")
private String userStatus;

public int getUserId() {
    return userId;
}

public void setUserId(int userId) {
    this.userId = userId;
}

public String getUserName() {
    return userName;
}

public void setUserName(String userName) {
    this.userName = userName;
}

public String getUserLogin() {
    return userLogin;
}

public void setUserLogin(String userLogin) {
    this.userLogin = userLogin;
}

public String getUserPassword() {
    return userPassword;
}

public void setUserPassword(String userPassword) {
```

```

        this.userPassword = userPassword;
    }

    public Timestamp getUserDate() {
        return userDate;
    }

    public void setUserDate(Timestamp userDate) {
        this.userDate = userDate;
    }

    public String getUserStatus() {
        return userStatus;
    }

    public void setUserStatus(String userStatus) {
        this.userStatus = userStatus;
    }

}

pom.xml – зависимости в проекте
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.example</groupId>
    <artifactId>demo4</artifactId>
    <version>1.0-SNAPSHOT</version>
    <name>demo4</name>

```

```

<properties>
  <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  <maven.compiler.target>1.8</maven.compiler.target>
  <maven.compiler.source>1.8</maven.compiler.source>
  <junit.version>5.8.2</junit.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.hibernate</groupId>
    <artifactId>hibernate-core</artifactId>
    <version>6.0.0.Final</version>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.29</version>
  </dependency>
</dependencies>

<build>
  <plugins>
  </plugins>

```

```
</build>
</project>
```

persistence.xml – подключение базы данных к Hibernate

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/persistence
http://xmlns.jcp.org/xml/ns/persistence/persistence_2_2.xsd"
    version="2.2">
    <persistence-unit name="default">

        <class>entity.CountryEntity</class>
        <properties>
            <property name="hibernate.connection.url"
value="jdbc:mysql://localhost:3306/symonscat"/>
            <property name="hibernate.connection.driver_class"
value="com.mysql.cj.jdbc.Driver"/>
            <property name="hibernate.connection.username" value="root"/>
            <property name="hibernate.connection.password" value="edskmtnnver2020"/>
        </properties>
    </persistence-unit>
</persistence>
```

style.css – пример создания div классов объекта в игровой зоне и зоне команд

...

```
.ground {  
    width: 50px;  
    height: 50px;  
    background-image: url('../images/bg.png');  
    display: inline-block;  
    z-index: 2;  
}
```

```
.cat {  
    width: 50px;  
    height: 50px;  
    background-image: url('../images/cat.png');  
    display: inline-block;  
    z-index: 2;  
}
```

```
.right {  
    width: 50px;  
    height: 50px;  
    background-image: url('../images/right.png');  
    display: inline-block;  
}
```

...